

Week 5 Linux Lab Report

Simulating and Analysing Security Governance Scenarios

Monitoring Auditing Controls and Executive Reporting

A hands-on study based on the Equifax data breach

By Oluwatosin Olajubu
Lab carried out: 5 to 8 October 2026
Environment: Kali Linux virtual machine with Docker

1. Summary

In 2017, the credit reporting company Equifax lost the personal records of well over 140 million people. The attackers got in through a known flaw in a piece of software called Apache Struts. A repair for that flaw had been available for months, but it had not been applied. This lab asked me to build a small copy of that situation, study what went wrong, add controls to fix it, and then create the kind of reports that managers and board members use to keep watch over security.

I completed all four parts of the lab on my personal Kali Linux virtual machine using Docker, which lets a computer run small, separate "containers" that behave like individual servers. The report follows the order of the lab sheet. Every one of the 128 screenshots is numbered, labelled and placed next to the step it shows, and Appendix D lists them all.

Results at a glance

Part	What I did	Main outcome
Part 1 Set up the environment	Installed Docker, built three containers (web server, database, monitoring), wrote a vulnerability scanner, a patch tool and a governance tracker.	The scanner found three problems: an unpatched Apache Struts web server (Critical), a database with a default password (High) and a web server that could talk straight to the database (Medium).
Part 2 Analyse governance failures	Ran a simulated attack, analysed the failures and compared them with Equifax.	Six governance failures were identified and matched against the Equifax case.
Part 3 Add controls	Ran a script that added five groups of controls, re-checked the system, and tested the database password by hand.	Controls were recorded. The database administrator password was changed and proved by direct tests (the default password is now rejected). The Struts and network results could not be proved (see Section 6.2).
Part 4 Metrics and reporting	Produced five trend charts, an executive dashboard, a board report, a metrics framework and a user guide.	All reporting files were created. The numbers in them are sample figures made by the lab script, not live measurements.

Four things to note:

- **The lab really does recreate the Equifax conditions.** There was a known, unrepaired flaw, a weak password and a network with no walls inside it. Policies existed on paper, but nothing made sure they were followed.
- **I had to fix several problems in the lab tools along the way.** Names did not resolve, a file format did not match, and the patch tool overwrote its own record. Each fix is explained in Section 8. Finding and fixing these problems was a useful lesson about how fragile real security tooling can be.

- **A scanner can give a false sense of safety.** After the controls were added, the scanner said the default database password was no longer accepted. A direct test showed that it still worked. I changed the password, tested again, and now have direct proof (Section 6.3).
- **The evidence for the other controls is weaker, and I say so.** The Struts server could not be reached by the final scan, and the patching in this lab is simulated, so I cannot claim that Struts was proved safe. Section 6.7 rates the evidence for every control, and Section 9 lists the limits of my results.

How this report is organised

Section	What it covers
2 and 3	The Equifax case, key words, and a description of the lab environment.
4 (Part 1)	Installing Docker, building the containers, the scanner, the patch tool and the governance tracker.
5 (Part 2)	The simulated attack, the six governance failures and the comparison with Equifax.
6 (Part 3)	The controls, how they were checked, the database password tests and how strong the evidence is.
7 (Part 4)	Metrics, the executive dashboard, five trend charts and the board-level reports.
8 to 11	Problems and fixes, limits of the evidence, recommendations and conclusion.
Appendices A to D	Files produced, a timeline of the lab, a checklist of deliverables and a list of all figures.

2. Background: what happened at Equifax

Equifax collects financial information about people, such as names, birth dates, addresses and card details. In March 2017¹, a serious flaw (named **CVE-2017-5638**) was made public in Apache Struts, a tool many companies use to build websites. A fix was released at the same time. Equifax did not apply the fix to one of its public web systems in time. Attackers used the flaw to get in, moved on to other systems and took data for about 76 days before anyone noticed.

The most important lesson is that this was not only a technology failure. It was a **governance** failure. Governance means the way an organisation sets rules, gives people clear duties, checks that the rules are followed and reports honestly to its leaders. At Equifax, the rules existed, but they were not carried out, the checks did not work and the right people were not held responsible.

¹ <https://oversight.house.gov/wp-content/uploads/2018/12/Equifax-Report.pdf>

A short timeline of the Equifax breach

When	What happened
7 March 2017	The Struts flaw (CVE-2017-5638) is made public, and a repaired version of the software is released at the same time.
March 2017	Equifax is warned about the flaw, but its own checks do not find the affected web system, so the repair is not applied.
Mid-May 2017	Attackers use the flaw to enter through the unrepaired web system and then move on to other systems.
29 July 2017	Equifax notices suspicious activity, about 76 days after the attackers got in, and acts to stop it.
7 September 2017	Equifax tells the public about the breach.
Afterwards	Reports say around 147 million people were affected. In 2019 Equifax agreed to a large settlement with United States regulators.

The dates above come from public reports and are given as a guide. The point to notice is the gap between March (a repair exists) and May (the attackers arrive). That gap is the part that good governance should have closed.

Key words used in this report

Word	Simple meaning
Governance	The rules, roles and checks that make sure security is managed properly and leaders can see how it is going.
Vulnerability	A weakness in software or settings that an attacker could use.
Patch	A repair that fixes a known weakness.
Container	A small, separate "mini computer" run by Docker. Each container does one job.
Network segmentation	Splitting a network into separate zones with controlled doors between them, so a break-in in one zone does not spread to all of them.
Default credentials	The user name and password a product comes with. Attackers try these first, so they must be changed.
Monitoring	Watching systems all the time so that suspicious activity is noticed and someone is alerted.
Metric	A number that shows how well something is working, such as the share of systems that are fully patched.
Dashboard	A single page that shows the key numbers and colours (green, amber, red) so leaders can see the position quickly.

3. The lab environment

The lab sheet suggested an Ubuntu 20.04 server. I used a **Kali Linux** virtual machine instead, because it was the system available to me. The tools were the same in practice: the terminal, Docker, Python 3 and the nano text editor. Docker was not installed at the start, so my first job was to install it (Section 4.1).

The finished environment had three containers on one Docker network called **secgov_network**:

Container	Job	Network address	Software
s2-045-struts2-1	Web server (the target with the known flaw)	172.19.0.2	vulhub/struts2:2.3.30
monitoring_server	Watching and reporting	172.19.0.3	ubuntu:20.04
database_server	Holds the customer records	172.19.0.4	mysql:5.7

Important note about the Struts image

The lab sheet names the web server image as **vulhub/struts2:s2-045**. In my lab, the web server that is actually running comes from the vulhub project folder called **s2-045**, and that folder uses the image **vulhub/struts2:2.3.30** (Struts version 2.3.30). The folder name describes the flaw being demonstrated (S2-045). The image name describes the Struts version.

Version 2.3.30 is older than the repaired versions named in the lab (2.3.32 and 2.5.10.1), so it is affected by CVE-2017-5638. The running container is named **s2-045-struts2-1**, not **web_server** as in the lab sheet. This difference is the reason I had to change names and addresses in the scripts later.

3.1 Three ways to reach a container

Many of the faults I met in this lab came from one idea: a container can be reached in more than one way, and each way works only from certain places. It helps to understand this before reading the rest of the report.

Method	Example	Works from	What happened in my lab
By name	s2-045-struts2-1	Only from inside the Docker network (for example from another container)	Failed in the first scan, because the scanner runs on the Kali machine, outside the network.
By network address	172.19.0.2:8080	Inside the Docker network. From Kali it is not always reachable.	Worked for the scanner on 7 October. Later it gave "Connection refused" and "No route to host".

Method	Example	Works from	What happened in my lab
By published port	localhost:8080	The Kali machine itself	Worked every time I tested it, so I used it for the attack simulation.

All customer records in the database are made-up examples supplied by the lab sheet. No real personal data was used.

4. Part 1: Setting up a simulated environment

4.1 Installing Docker

I made the lab folder (**security_governance_lab**) and tried to create the Docker network. The terminal replied that the command **docker** was not found, so Docker had to be installed first. I installed the Docker command tool, then the full Docker engine, started the service and tested it with the standard "hello-world" container.

```

kali@kali: ~/security_governance_lab
Session Actions Edit View Help
—(kali@kali)-[~]
$ gsettings set org.gnome.desktop.interface cursor-blink true

—(kali@kali)-[~]
$ mkdir -p ~/security_governance_lab

—(kali@kali)-[~]
$ cd ~/security_governance_lab

—(kali@kali)-[~/security_governance_lab]
$ docker network create secgov_network
command 'docker' not found, but can be installed with:
sudo apt install docker-cli
sudo apt install podman-docker

—(kali@kali)-[~/security_governance_lab]
$ 

```

Figure 1: Creating the lab folder. The attempt to create the Docker network fails because Docker is not installed.

```
Session  Actions  Edit  View  Help
(kali@kali)-[~]
$ gsettings set org.gnome.desktop.interface cursor-blink true
(kali@kali)-[~]
$ mkdir -p ~/security_governance_lab
(kali@kali)-[~]
$ cd ~/security_governance_lab
(kali@kali)-[~/security_governance_lab]
$ docker network create secgov_network
Command 'docker' not found, but can be installed with:
sudo apt install docker-cli
sudo apt install podman-docker
(kali@kali)-[~/security_governance_lab]
$ sudo apt install docker-cli
[sudo] password for kali:
The following package was automatically installed and is no longer required:
  libcrypt-dev
Use 'sudo apt autoremove' to remove it.

Installing:
  docker-cli

Installing dependencies:
  docker-buildx
```

Figure 2: Installing the Docker command tool (docker-cli) after the "command not found" message.

```
Suggested packages:
  docker-doc

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 2240
  Download size: 22.4 MB
  Space needed: 104 MB / 61.3 GB available

Continue? [Y/n] Y
Get:1 http://http.kali.org/kali kali-rolling/main amd64 docker-buildx amd64 0
.29.1+ds1-4 [14.3 MB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 docker-cli amd64 28.5
.2+dfsg4-4 [8,049 kB]
Fetched 22.4 MB in 16s (1,426 kB/s)
Selecting previously unselected package docker-buildx.
(Reading database ... 423747 files and directories currently installed.)
Preparing to unpack .../docker-buildx_0.29.1+ds1-4_amd64.deb ...
Unpacking docker-buildx (0.29.1+ds1-4) ...
Selecting previously unselected package docker-cli.
Preparing to unpack .../docker-cli_28.5.2+dfsg4-4_amd64.deb ...
Unpacking docker-cli (28.5.2+dfsg4-4) ...
Setting up docker-cli (28.5.2+dfsg4-4) ...
Setting up docker-buildx (0.29.1+ds1-4) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...
```

Figure 3: The Docker command tool being downloaded and unpacked (22.4 megabytes).

```
(kali@kali)-[~/security_governance_lab]
$ sudo apt install docker.io
The following package was automatically installed and is no longer required:
  libcrypt-dev
Use 'sudo apt autoremove' to remove it.

Installing:
  docker.io

Installing dependencies:
  containerd      libcompel1      python3-pycriu  tini-static
  criu            python3-protobuf  runc

Suggested packages:
  containernetworking-plugins  debootstrap  xfsprogs
  docker-doc                   rinse        zfs-fuse
  btrfs-progs                  rootlesskit  | zfsutils-linux

Recommended packages:
  needrestart

Summary:
  Upgrading: 0, Installing: 8, Removing: 0, Not Upgrading: 2246
  Download size: 56.4 MB
  Space needed: 229 MB / 61.2 GB available

Continue? [Y/n] Y
```

Figure 4: Installing the full Docker engine package (docker.io) together with the packages it needs.

```
Selecting previously unselected package python3-protobuf.
Preparing to unpack .../6-python3-protobuf_3.21.12-16_amd64.deb ...
Unpacking python3-protobuf (3.21.12-16) ...
Selecting previously unselected package python3-pycriu.
Preparing to unpack .../7-python3-pycriu_4.2.1-1_all.deb ...
Unpacking python3-pycriu (4.2.1-1) ...
Setting up tini-static (0.19.0-6+b2) ...
Setting up runc (1.3.6+ds1-1) ...
Setting up libcompel1:amd64 (4.2.1-1) ...
Setting up criu (4.2.1-1) ...
Setting up python3-protobuf (3.21.12-16) ...
Setting up containerd (2.1.9+ds1-3) ...
containerd.service is a disabled or a static unit, not starting it.
Setting up python3-pycriu (4.2.1-1) ...
Setting up docker.io (28.5.2+dfsg4-4) ...
Creating group 'docker' with GID 982.
update-rc.d: We have no instructions for the docker init script.
update-rc.d: It looks like a non-network service, we enable it.
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service'
→ '/usr/lib/systemd/system/docker.service'.
Created symlink '/etc/systemd/system/sockets.target.wants/docker.socket' → '/
usr/lib/systemd/system/docker.socket'.
Processing triggers for kali-menu (2025.4.3) ...
Processing triggers for libc-bin (2.42-16) ...
Processing triggers for man-db (2.13.1-1) ...
```

Figure 5: The Docker engine installation finishing.

```

└─$ sudo systemctl start docker

(kali@kali)-[~/security_governance_lab]
└─$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset>
   Active: active (running) since Mon 2026-10-05 18:06:49 EDT; 2min 48s ago
  Invocation: 2c30d7aecf9743e9a400f89da4db672c
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 18148 (dockerd)
     Tasks: 10
    Memory: 28M (peak: 31.4M)
       CPU: 642ms
    CGroup: /system.slice/docker.service
            └─18148 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd>

Oct 05 18:06:48 kali dockerd[18148]: time="2026-10-05T18:06:48.412982914-04:>
Oct 05 18:06:48 kali dockerd[18148]: time="2026-10-05T18:06:48.487660618-04:>
Oct 05 18:06:48 kali dockerd[18148]: time="2026-10-05T18:06:48.544759955-04:>
Oct 05 18:06:49 kali dockerd[18148]: time="2026-10-05T18:06:49.259720176-04:>
Oct 05 18:06:49 kali dockerd[18148]: time="2026-10-05T18:06:49.288934616-04:>
Oct 05 18:06:49 kali dockerd[18148]: time="2026-10-05T18:06:49.289125267-04:>
Oct 05 18:06:49 kali dockerd[18148]: time="2026-10-05T18:06:49.316098017-04:>
Oct 05 18:06:49 kali dockerd[18148]: time="2026-10-05T18:06:49.332939445-04:>
Oct 05 18:06:49 kali dockerd[18148]: time="2026-10-05T18:06:49.333169551-04:>
Oct 05 18:06:49 kali systemd[1]: Started docker.service - Docker Application>
lines 1-23/23 (FNN)

```

Figure 6: Starting the Docker service and confirming that it is active and running.

```

(kali@kali)-[~/security_governance_lab]
└─$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent i
    t
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

```

Figure 7: The "hello-world" test container runs. Docker reports that the installation is working.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
(kali@kali)-[~]
$ groups
kali adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin wireshark kaboxer docker
(kali@kali)-[~]
$ cd ~/security_governance_lab
(kali@kali)-[~/security_governance_lab]
$ ls ~
Desktop          lynis-audit.txt
Documents        Music
Downloads        Pictures
GRC101_C11_26_cgrce_17602.tar.gz  Public
GRC101_Lab       security_governance_lab
GRC101_Lab02_Command_Log  Videos
(kali@kali)-[~/security_governance_lab]
$ docker network create secgov_network
27575bab0c26a10f912b82ee45ffdf6cdc73980d8b30c7a1348359d9cde0fba4
(kali@kali)-[~/security_governance_lab]
$
```

Figure 8: Docker commands now run correctly, and the lab network (secgov_network) is created.

4.2 Creating the lab files

Next I wrote the files that describe the lab. The **docker-compose.yml** file lists the containers to build. The **init.sql** file creates a **customers** table in the database and adds three made-up customers. The table deliberately holds the kind of sensitive details Equifax lost: social security number, date of birth, address, email, phone number and card number.

```

    MYSQL_DATABASE: customer_data
    MYSQL_USER: app_user
    MYSQL_PASSWORD: app_password
  volumes:
    - ./database init:/docker-entrypoint-initdb.d
  networks:
    - secgov_network

monitoring:
  image: ubuntu:20.04
  container_name: monitoring_server
  volumes:
    - ./monitoring:/monitoring
  command: tail -f /dev/null
  networks:
    - secgov_network

networks:
  secgov_network:
    external: true
EOF

(kali@kali)-[~/security_governance_lab]
$ ls
docker-compose.yml

```

Figure 9: The end of docker-compose.yml: the monitoring container and the network setting, closed with the EOF marker.

```

(kali@kali)-[~/security_governance_lab]
$ >....
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  ssn VARCHAR(11),
  dob DATE,
  address VARCHAR(100),
  city VARCHAR(50),
  state VARCHAR(2),
  zip VARCHAR(10),
  email VARCHAR(100),
  phone VARCHAR(15),
  credit_card VARCHAR(16)
);

INSERT INTO customers (first_name, last_name, ssn, dob, address, city, state,
zip, email, phone, credit_card)
VALUES
('John', 'Doe', '123-45-6789', '1980-01-15', '123 Main St', 'Anytown', 'CA',
'12345', 'john.doe@example.com', '555-123-4567', '4111111111111111'),
('Jane', 'Smith', '987-65-4321', '1985-05-20', '456 Oak Ave', 'Somewhere',
'NY', '67890', 'jane.smith@example.com', '555-987-6543', '5555555555554444'),

```

Figure 10: Writing init.sql: the customers table and its columns, including social security number and card number.

```

INSERT INTO customers (first_name, last_name, ssn, dob, address, city, state,
zip, email, phone, credit_card)
VALUES
('John', 'Doe', '123-45-6789', '1980-01-15', '123 Main St', 'Anytown', 'CA',
'12345', 'john.doe@example.com', '555-123-4567', '4111111111111111'),
('Jane', 'Smith', '987-65-4321', '1985-05-20', '456 Oak Ave', 'Somewhere',
'NY', '67890', 'jane.smith@example.com', '555-987-6543', '5555555555554444'),

('Bob', 'Johnson', '456-78-9012', '1975-11-30', '789 Pine Rd', 'Nowhere',
TX', '54321', 'bob.johnson@example.com', '555-456-7890', '3782822463100005');

EOF

(kali㉿kali)-[~/security_governance_lab]
$

```

Figure 11: The three made-up customer records added to the table.

```

(kali㉿kali)-[~/security_governance_lab]
$ ls database_init
init.sql

(kali㉿kali)-[~/security_governance_lab]
$

```

Figure 12: Checking that init.sql exists inside the database_init folder.

```

(kali㉿kali)-[~/security_governance_lab]
$ ls database_init
init.sql

(kali㉿kali)-[~/security_governance_lab]
$ mkdir -p monitoring

(kali㉿kali)-[~/security_governance_lab]
$ ls
database_init  docker-compose.yml  monitoring

(kali㉿kali)-[~/security_governance_lab]
$ ls database_init
init.sql

(kali㉿kali)-[~/security_governance_lab]
$

```

Figure 13: The lab folder now contains database_init, monitoring and docker-compose.yml.

```

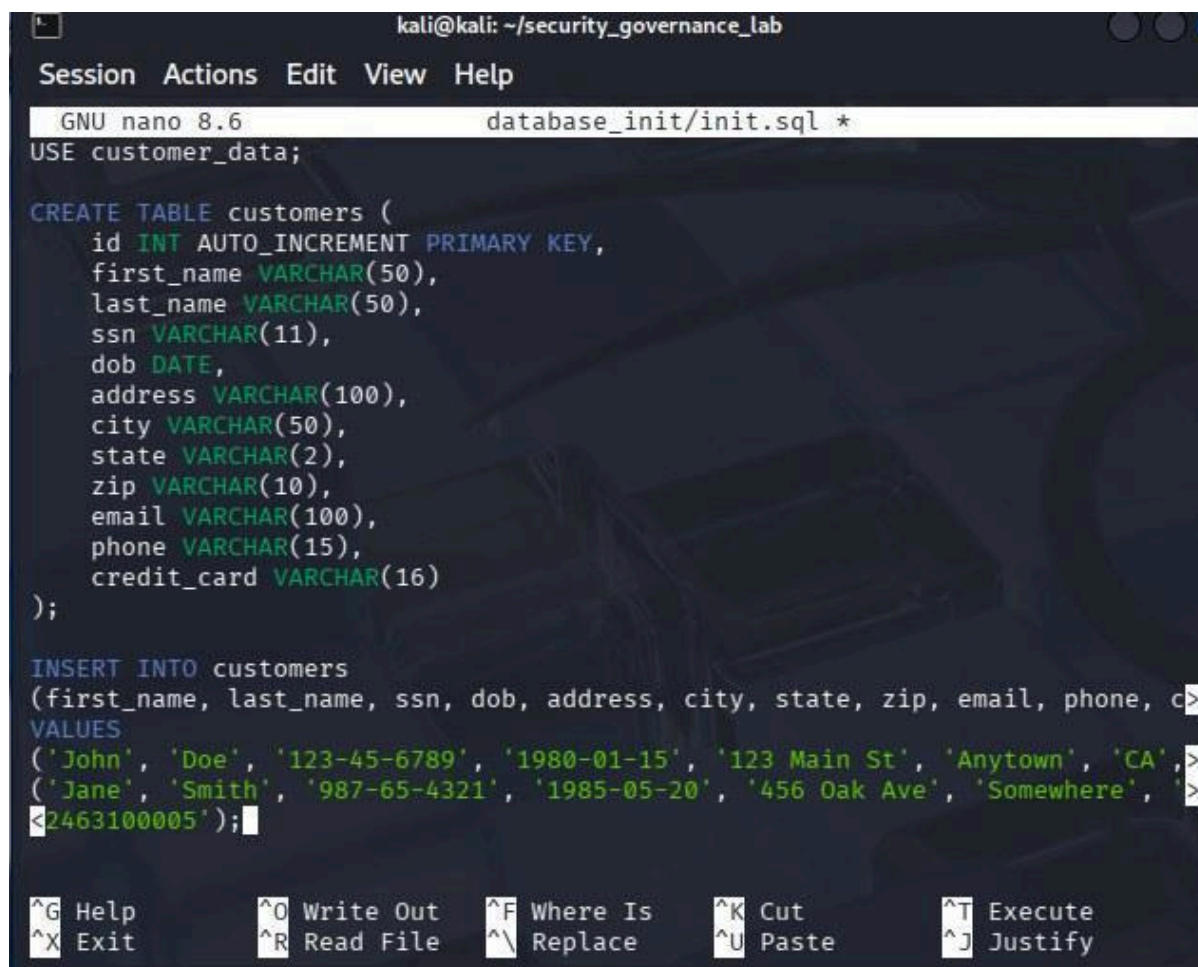
$ mkdir -p database_init monitoring

(kali㉿kali)-[~/security_governance_lab]
$ nano database_init/init.sql

(kali㉿kali)-[~/security_governance_lab]
$

```

Figure 14: Creating the database_init and monitoring folders and opening init.sql in the nano editor.

A screenshot of a terminal window with the nano text editor open. The title bar shows 'kali@kali: ~/security_governance_lab'. The editor's menu bar includes 'Session', 'Actions', 'Edit', 'View', and 'Help'. The status bar at the top indicates 'GNU nano 8.6' and 'database_init/init.sql *'. The main text area contains SQL code: 'USE customer_data;', 'CREATE TABLE customers (' followed by a list of columns with their data types and constraints, and 'INSERT INTO customers' with two rows of sample data. The bottom status bar shows various keyboard shortcuts like '^G Help', '^O Write Out', '^F Where Is', '^K Cut', '^T Execute', '^X Exit', '^R Read File', '^_ Replace', '^U Paste', and '^J Justify'.

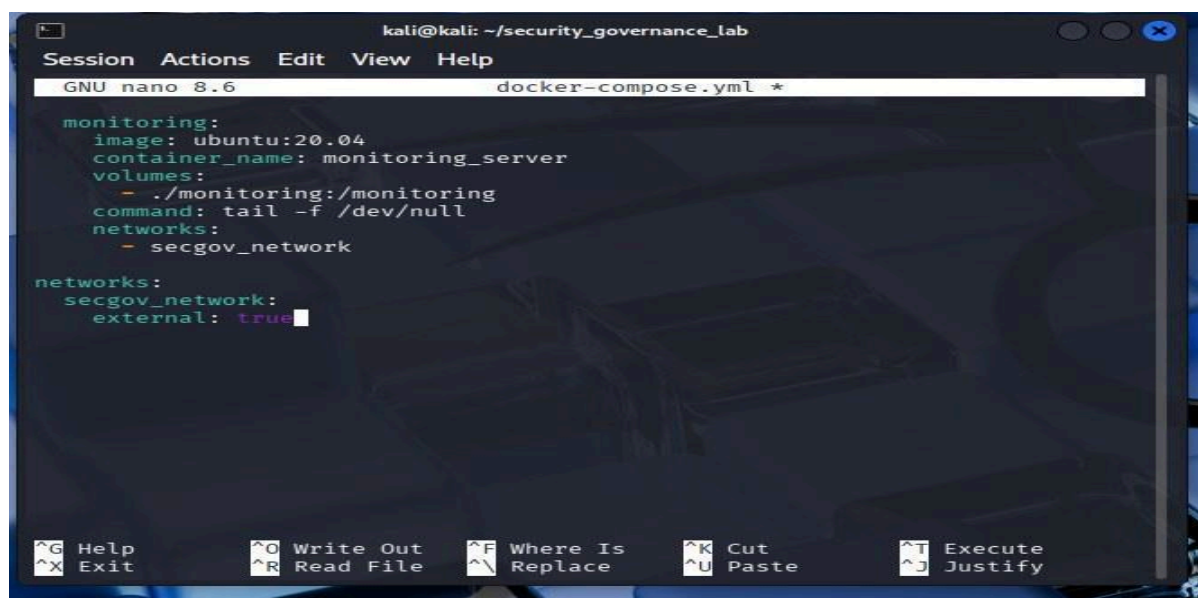
```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 database_init/init.sql *
USE customer_data;

CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  ssn VARCHAR(11),
  dob DATE,
  address VARCHAR(100),
  city VARCHAR(50),
  state VARCHAR(2),
  zip VARCHAR(10),
  email VARCHAR(100),
  phone VARCHAR(15),
  credit_card VARCHAR(16)
);

INSERT INTO customers
(first_name, last_name, ssn, dob, address, city, state, zip, email, phone, c
VALUES
('John', 'Doe', '123-45-6789', '1980-01-15', '123 Main St', 'Anytown', 'CA',
('Jane', 'Smith', '987-65-4321', '1985-05-20', '456 Oak Ave', 'Somewhere',
2463100005');

^G Help      ^O Write Out  ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File  ^_ Replace   ^U Paste     ^J Justify
```

Figure 15: Reviewing init.sql in the nano editor.

A screenshot of a terminal window with the nano text editor open. The title bar shows 'kali@kali: ~/security_governance_lab'. The editor's menu bar includes 'Session', 'Actions', 'Edit', 'View', and 'Help'. The status bar at the top indicates 'GNU nano 8.6' and 'docker-compose.yml *'. The main text area contains YAML configuration for a 'monitoring' service and a 'secgov_network' network. The bottom status bar shows various keyboard shortcuts like '^G Help', '^O Write Out', '^F Where Is', '^K Cut', '^T Execute', '^X Exit', '^R Read File', '^_ Replace', '^U Paste', and '^J Justify'.

```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 docker-compose.yml *

monitoring:
  image: ubuntu:20.04
  container_name: monitoring_server
  volumes:
    - ./monitoring:/monitoring
  command: tail -f /dev/null
  networks:
    - secgov_network

networks:
  secgov_network:
    external: true

^G Help      ^O Write Out  ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File  ^_ Replace   ^U Paste     ^J Justify
```

Figure 16: Reviewing the monitoring section of docker-compose.yml in the nano editor.

4.3 Problem: the compose tool was missing

Before starting the containers, I checked my compose file with **docker-compose config**. This failed. The old command was not installed, the newer **docker compose** command was

unknown, and the compose plug-in package was not available from the Kali software list. I solved this by downloading Docker Compose version 2 as a standalone file, placing it in Docker's plug-in folder and making it executable. After that, **docker compose config** worked. It showed a harmless warning that the "version" line in the file is out of date.

```
(kali@kali)-[~/security_governance_lab]
$ docker-compose config
Command 'docker-compose' not found, but can be installed with:
sudo apt install docker-compose
Do you want to install it? (N/y)docker^C

(kali@kali)-[~/security_governance_lab]
$ docker compose version
docker: unknown command: docker compose

Run 'docker --help' for more information

(kali@kali)-[~/security_governance_lab]
$ sudo apt update
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2246 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~/security_governance_lab]
$ sudo apt install docker-compose-plugin
Package docker-compose-plugin is not available, but is referred to by another
package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

Error: Package 'docker-compose-plugin' has no installation candidate
```

Figure 17: The compose check fails: the old command is not found, the new command is unknown and the plug-in package is not available.

```
(kali@kali)-[~/security_governance_lab]
$ sudo mkdir -p /usr/local/lib/docker/cli-plugins

(kali@kali)-[~/security_governance_lab]
$ sudo curl -SL https://github.com/docker/compose/releases/latest/download/
docker-compose-linux-x86_64 \
-o /usr/local/lib/docker/cli-plugins/docker-compose
```

	% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
				Dload	Upload	Total	Spent	Left
0	0	0	0	0	0	--:--:--	0:00:01	--:--:--
0	0	0	0	0	0	--:--:--	0:00:01	--:--:--
1	31897k	1 602784	0	118071	0	0:04:36	0:00:05	0:04:31
7	31897k	7 2540k	0	424356	0	0:01:16	0:00:06	0:01:10
16	31897k	16 5324k	0	768195	0	0:00:42	0:00:07	0:00:35
26	31897k	26 8588k	0	1060k	0	0:00:30	0:00:08	0:00:22
37	31897k	37 12111k	0	1331k	0	0:00:23	0:00:09	0:00:14
48	31897k	48 15401k	0	1521k	0	0:00:20	0:00:10	0:00:10
61	31897k	61 19736k	0	1778k	0	0:00:17	0:00:11	0:00:06
76	31897k	76 24271k	0	1988k	0	0:00:16	0:00:12	0:00:04
91	31897k	91 29215k	0	2226k	0	0:00:14	0:00:13	0:00:01
100	31897k	100 31897k	0	2346k	0	0:00:13	0:00:13	--:--:--

```
7k
```

Figure 18: Downloading Docker Compose version 2 as a standalone file into Docker's plug-in folder.

```
(kali@kali)-[~/security_governance_lab]
$ sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

(kali@kali)-[~/security_governance_lab]
$ docker compose version
Docker Compose version v5.6.0

(kali@kali)-[~/security_governance_lab]
$ docker compose config
WARN[0000] /home/kali/security_governance_lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
name: security_governance_lab
services:
  database:
    container_name: database_server
    environment:
      MYSQL_DATABASE: customer_data
      MYSQL_PASSWORD: app_password
      MYSQL_ROOT_PASSWORD: password
      MYSQL_USER: app_user
    image: mysql:5.7
    networks:
      secgov_network: null
    volumes:
      - type: bind
        source: /home/kali/security_governance_lab/database_init
```

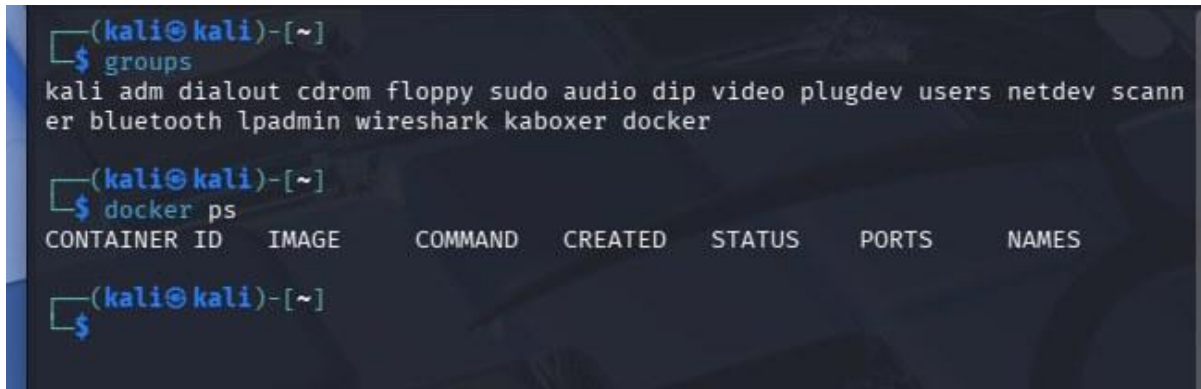
Figure 19: The tool is made executable and now reports its version. The configuration check prints the database service and a warning about the out-of-date "version" line.

```
monitoring:
  container_name: monitoring_server
  command:
    - tail
    - -f
    - /dev/null
  image: ubuntu:20.04
  networks:
    secgov_network: null
  volumes:
    - type: bind
      source: /home/kali/security_governance_lab/monitoring
      target: /monitoring
      bind: {}
web:
  container_name: web_server
  image: vulhub/struts2:s2-045
  networks:
    secgov_network: null
  ports:
    - mode: ingress
      target: 8080
      published: "8080"
      protocol: tcp
networks:
  secgov_network:
    name: secgov_network
    external: true
```

Figure 20: The rest of the configuration check: the monitoring and web services. The web service still points to the image named in the lab sheet (vulhub/struts2:s2-045).

4.4 Fixing Docker permissions

Docker commands at first needed **sudo** (administrator rights). I added my user to the **docker** group so that commands such as **docker ps** work without it. The screenshot shows that the group is present and that **docker ps** now runs normally.



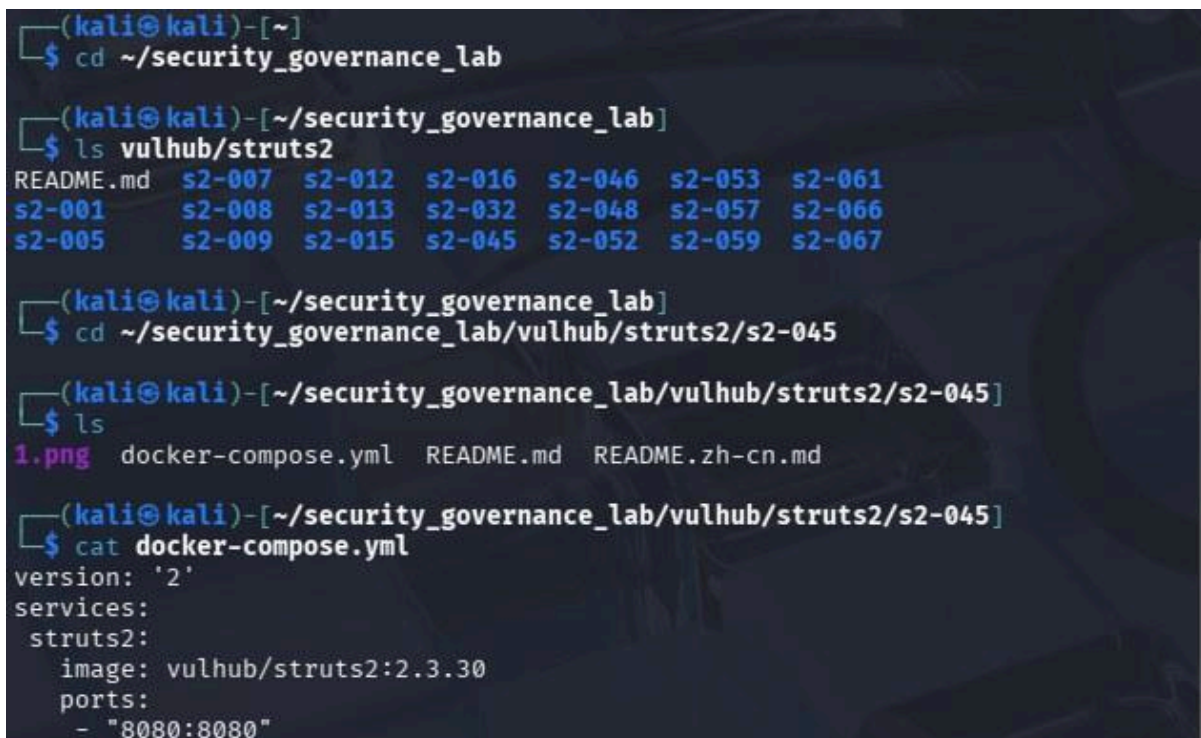
```
(kali㉿kali)-[~]
$ groups
kali adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin wireshark kaboxer docker

(kali㉿kali)-[~]
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
```

Figure 21: The docker group appears in the user's groups, and docker ps runs without administrator rights.

4.5 Starting the vulnerable web server

I used the ready-made Struts environment from the vulhub project. Its folder is called **s2-045** and it uses the image **vulhub/struts2:2.3.30**, as explained in Section 3. I pulled the image, installed a compose package that worked on this system and started the container. My first start command contained a mistyped option, which Docker rejected; I corrected it and the container started.



```
(kali㉿kali)-[~]
$ cd ~/security_governance_lab

(kali㉿kali)-[~/security_governance_lab]
$ ls vulhub/struts2
README.md  s2-007  s2-012  s2-016  s2-046  s2-053  s2-061
s2-001     s2-008  s2-013  s2-032  s2-048  s2-057  s2-066
s2-005     s2-009  s2-015  s2-045  s2-052  s2-059  s2-067

(kali㉿kali)-[~/security_governance_lab]
$ cd ~/security_governance_lab/vulhub/struts2/s2-045

(kali㉿kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ ls
1.png  docker-compose.yml  README.md  README.zh-cn.md

(kali㉿kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ cat docker-compose.yml
version: '2'
services:
  struts2:
    image: vulhub/struts2:2.3.30
    ports:
      - "8080:8080"
```

Figure 22: The vulhub project folder for s2-045. Its own docker-compose.yml uses the image vulhub/struts2:2.3.30 and publishes port 8080.

```

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker pull vulhub/struts2:2.3.30
2.3.30: Pulling from vulhub/struts2
092586df9206: Pull complete
ef599477fae0: Pull complete
4530c6472b5d: Pull complete
d34d61487075: Pull complete
272f46008219: Pull complete
12ff6ccfe7a6: Pull complete
f26b99e1adb1: Pull complete
7a1b276fce42: Pull complete
c944b4612eac: Pull complete
293fc5ed12f1: Pull complete
555a865b2eb3: Pull complete
86d6abac5e19: Pull complete
Digest: sha256:b992e080d8e5ad332bf526ae26124d28f56ca4fc9e0eaa91510dbee9870eb2c8
Status: Downloaded newer image for vulhub/struts2:2.3.30
docker.io/vulhub/struts2:2.3.30

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$

```

Figure 23: Pulling the image vulhub/struts2:2.3.30.

```

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker-compose --version
Command 'docker-compose' not found, but can be installed with:
sudo apt install docker-compose
Do you want to install it? (N/y)y
sudo apt install docker-compose
[sudo] password for kali:
The following package was automatically installed and is no longer required:
  libcrypt-dev
Use 'sudo apt autoremove' to remove it.

Installing:
  docker-compose

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2254
  Download size: 13.9 MB
  Space needed: 69.4 MB / 60.2 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 docker-compose amd64
2.40.3-3 [13.9 MB]
Fetched 13.9 MB in 14s (981 kB/s)
Selecting previously unselected package docker-compose.
(Reading database ... 424312 files and directories currently installed.)

```

Figure 24: The docker-compose command is not found, so the docker-compose package is installed from the Kali software list.

```

Installing:
  docker-compose

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2254
  Download size: 13.9 MB
  Space needed: 69.4 MB / 60.2 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 docker-compose amd64
2.40.3-3 [13.9 MB]
Fetched 13.9 MB in 14s (981 kB/s)
Selecting previously unselected package docker-compose.
(Reading database ... 424312 files and directories currently installed.)
Preparing to unpack ... /docker-compose_2.40.3-3_amd64.deb ...
Unpacking docker-compose (2.40.3-3) ...
Setting up docker-compose (2.40.3-3) ...
Processing triggers for kali-menu (2025.4.3) ...
Processing triggers for man-db (2.13.1-1) ...

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker-compose --version
Docker Compose version 2.40.3-3

```

Figure 25: The docker-compose package finishes installing.

```

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker-compose --version
Docker Compose version 2.40.3-3

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker-compose up -dn
unknown shorthand flag: 'n' in -n

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker-compose up -d
WARN[0000] /home/kali/security_governance_lab/vulhub/struts2/s2-045/docker-co
mpose.yml: the attribute `version` is obsolete, it will be ignored, please re
move it to avoid potential confusion
[+] Running 2/2
 ✓ Network s2-045_default      Created      0.1s
 ✓ Container s2-045-struts2-1  Started    0.8s

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
fc6987ddd75c   vulhub/struts2:2.3.30              "/usr/local/bin/mvn-..." 56 seconds ago Up 54 seconds 0.0.0.0:8080→8080/tcp, [::]:8080→8080/tcp s2-045-stru
ts2-1

```

Figure 26: The Struts container starts. After a mistyped option is corrected, docker ps shows the container running.

Evidence of the running web server:

Item	Value
Image	vulhub/struts2:2.3.30
Status	Up 54 seconds (at the time of the screenshot)
Ports	0.0.0.0:8080 to 8080 (reachable on port 8080 of this machine)

Item	Value
Container name	s2-045-struts2-1

A controlled, deliberately vulnerable Struts 2.3.30 web application was therefore deployed inside an isolated Docker lab, offering its service on port 8080. To test it, I opened the address in the web browser on the virtual machine. The page shown is the Struts "Fileupload sample", which confirms the application is running.

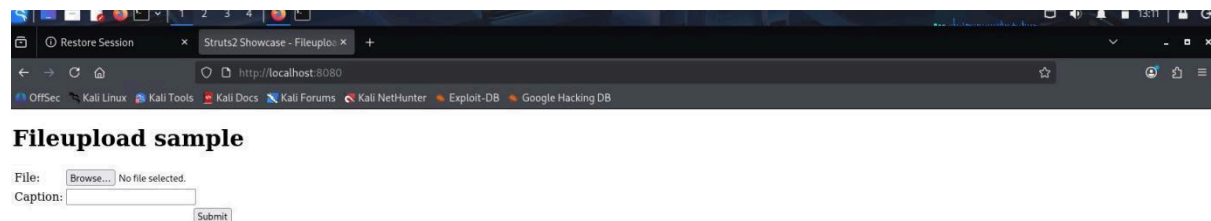


Figure 27: The Struts test application opened in the browser on the virtual machine: the "Fileupload sample" page.

```
(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ pwd
ls -la
/home/kali/security_governance_lab/vulhub/struts2/s2-045
total 88
drwxrwxr-x 2 kali kali 4096 Oct 7 12:27 .
drwxrwxr-x 22 kali kali 4096 Oct 7 12:27 ..
-rw-rw-r-- 1 kali kali 65726 Oct 7 12:27 1.png
-rw-rw-r-- 1 kali kali 93 Oct 7 12:27 docker-compose.yml
-rw-rw-r-- 1 kali kali 1169 Oct 7 12:27 README.md
-rw-rw-r-- 1 kali kali 1048 Oct 7 12:27 README.zh-cn.md

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ docker ps -a
CONTAINER ID   IMAGE          PORTS          COMMAND          CREATED          STATUS          PORTS
fc6987ddd75c   vulhub/struts2:2.3.30   "/usr/local/bin/mvn-..."   26 minutes ago   Up 26 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
2-045-struts2-1
3bf90d65a2dd   hello-world    "/hello"       43 hours ago     Exited (0) 43 hours ago
ad_banach
```

Figure 28: Folder listing and docker ps -a: the Struts container is up and running, next to the older hello-world test container.

```

(kali@kali)-[~/security_governance_lab/vulhub/struts2/s2-045]
$ cd ~/security_governance_lab

(kali@kali)-[~/security_governance_lab]
$ ls -la
total 12
drwxrwxr-x  3 kali kali 4096 Oct  7 12:25 .
drwx----- 22 kali kali 4096 Oct  7 12:37 ..
drwxrwxr-x 158 kali kali 4096 Oct  7 12:27 vulhub

```

Figure 29: Moving between folders to return to the main lab folder.

4.6 Starting the database and monitoring containers and checking the network

I created the network **secgov_network** and listed the networks to confirm that it exists. I then started the database and monitoring containers with **docker compose up -d database monitoring**. A listing of running containers showed all three servers up. I read the database log to confirm that MySQL had started. The log also contained a warning that an old, weak setting for encrypted connections (an outdated version of the protocol that protects network traffic) was still switched on. This is a small example of an ordinary weakness that a governance process should notice.

To check that the database worked, I logged in as the application user and listed the tables, which showed the **customers** table. Finally I inspected the network to see which address each container received. This was important later, because the scanner must be given the right address.

```

(kali@kali)-[~/security_governance_lab]
$ cd ~/security_governance_lab

(kali@kali)-[~/security_governance_lab]
$ mkdir -p database_init monitoring

(kali@kali)-[~/security_governance_lab]
$ docker network create secgov_network
f3ab237558c1e78293f4104cc0e0eb77db3ef3345a3d1b77b082f5a53ef9ade9

(kali@kali)-[~/security_governance_lab]
$ docker network ls

```

NETWORK ID	NAME	DRIVER	SCOPE
a7c04aabd3c1	bridge	bridge	local
1af5d384ebce	host	host	local
02d23c3339f9	none	null	local
b8b88ce9df23	s2-045_default	bridge	local
f3ab237558c1	secgov_network	bridge	local

```

(kali@kali)-[~/security_governance_lab]
$ cd ~/security_governance_lab

(kali@kali)-[~/security_governance_lab]
$ mkdir -p database_init monitoring

```

Figure 30: Creating **secgov_network** and listing the Docker networks. The new network sits beside the default one created for the Struts container.

```

(kali@kali)-[~/security_governance_lab]
$ nano docker-compose.yml

(kali@kali)-[~/security_governance_lab]
$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
NAME          IMAGE                                PORTS
s2-045-struts2-1 vulhub/struts2:2.3.30             0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp

(kali@kali)-[~/security_governance_lab]
$ docker inspect -f '{{json .NetworkSettings.Networks}}' s2-045-struts2-1
{"s2-045_default":{"IPAMConfig":null,"Links":null,"Aliases":["s2-045-struts2-1","struts2"],"MacAddress":"ae:f0:21:89:e3:64","DriverOpts":null,"GwPriority":0,"NetworkID":"b8b88ce9df239fdaab0e16a9727a72f1149300ff557346a351f3a406aable462","EndpointID":"f9e24028a464a497975fcceb7c4265fdd3d6760041e1236da46014eb24995920","Gateway":"172.18.0.1","IPAddress":"172.18.0.2","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLen":0,"DNSNames":["s2-045-struts2-1","struts2","fc6987ddd75c"]}}

(kali@kali)-[~/security_governance_lab]
$ docker network connect secgov_network s2-045-struts2-1

(kali@kali)-[~/security_governance_lab]
$ docker inspect -f '{{json .NetworkSettings.Networks}}' s2-045-struts2-1
{"s2-045_default":{"IPAMConfig":null,"Links":null,"Aliases":["s2-045-struts2-1","struts2"],"MacAddress":"ae:f0:21:89:e3:64","DriverOpts":null,"GwPriority":0,"NetworkID":"b8b88ce9df239fdaab0e16a9727a72f1149300ff557346a351f3a406aable462","EndpointID":"f9e24028a464a497975fcceb7c4265fdd3d6760041e1236da46014eb24995920","Gateway":"172.18.0.1","IPAddress":"172.18.0.2","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLen":0,"DNSNames":["s2-045-struts2-1","struts2","fc6987ddd75c"]},"secgov_network":{"IPAMConfig":{"Driver":"bridge","Options":{"mac-address":"3e:64:14:af:b4:81"},"Subnet":"172.19.0.0/24"},"Links":null,"Aliases":["s2-045-struts2-1","struts2","fc6987ddd75c"],"MacAddress":"3e:64:14:af:b4:81","DriverOpts":{"mac-address":"3e:64:14:af:b4:81"},"GwPriority":0,"NetworkID":"f3ab237558c1e78293f4104cc0e0eb77db3ef3345a3d1b77b082f5a53ef9ade9","EndpointID":"5df0c7eea3c32f15d4ac0efb094714302074678a0b717c67f9feef4548578c4f","Gateway":"172.19.0.1","IPAddress":"172.19.0.2","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLen":0,"DNSNames":["s2-045-struts2-1","struts2","fc6987ddd75c"]}}

```

Figure 31: A formatted container list shows the Struts container, and docker inspect starts to show its networks.

```

(kali@kali)-[~/security_governance_lab]
$ docker inspect -f '{{json .NetworkSettings.Networks}}' s2-045-struts2-1
{"s2-045_default":{"IPAMConfig":null,"Links":null,"Aliases":["s2-045-struts2-1","struts2"],"MacAddress":"ae:f0:21:89:e3:64","DriverOpts":null,"GwPriority":0,"NetworkID":"b8b88ce9df239fdaab0e16a9727a72f1149300ff557346a351f3a406aable462","EndpointID":"f9e24028a464a497975fcceb7c4265fdd3d6760041e1236da46014eb24995920","Gateway":"172.18.0.1","IPAddress":"172.18.0.2","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLen":0,"DNSNames":["s2-045-struts2-1","struts2","fc6987ddd75c"]},"secgov_network":{"IPAMConfig":{"Driver":"bridge","Options":{"mac-address":"3e:64:14:af:b4:81"},"Subnet":"172.19.0.0/24"},"Links":null,"Aliases":["s2-045-struts2-1","struts2","fc6987ddd75c"],"MacAddress":"3e:64:14:af:b4:81","DriverOpts":{"mac-address":"3e:64:14:af:b4:81"},"GwPriority":0,"NetworkID":"f3ab237558c1e78293f4104cc0e0eb77db3ef3345a3d1b77b082f5a53ef9ade9","EndpointID":"5df0c7eea3c32f15d4ac0efb094714302074678a0b717c67f9feef4548578c4f","Gateway":"172.19.0.1","IPAddress":"172.19.0.2","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLen":0,"DNSNames":["s2-045-struts2-1","struts2","fc6987ddd75c"]}}

```

Figure 32: Docker inspect output: the Struts container has an address on its own network (172.18.0.2) and another on secgov_network (172.19.0.2).

```

(kali@kali)-[~/security_governance_lab]
$ docker compose up -d database monitoring
[+] Running 14/14
  ✓ monitoring Pulled                                39.0s
  ✓ 13b7e930469f Pull complete                       24.4s
  ✓ database Pulled                                  81.6s
  ✓ 20e4dcae4c69 Pull complete                       39.4s
  ✓ 1c56c3d4ce74 Pull complete                       39.5s
  ✓ e9f03a1c24ce Pull complete                       39.6s
  ✓ 68c3898c2015 Pull complete                       40.6s
  ✓ 6b95a940e7b6 Pull complete                       40.7s
  ✓ 90986bb8de6e Pull complete                       40.7s
  ✓ ae71319cb779 Pull complete                       43.7s
  ✓ ffc89e9dfd88 Pull complete                       43.8s
  ✓ 43d05e938198 Pull complete                       66.9s
  ✓ 064b2d298fba Pull complete                       66.9s
  ✓ df9a4d85569b Pull complete                       67.0s
[+] Running 2/2
  ✓ Container database_server Started                1.7s
  ✓ Container monitoring_server Started...            1.7s

```

Figure 33: Starting the database and monitoring containers. Docker pulls the images and starts both.

```
(kali@kali)-[~/security_governance_lab]
$ docker ps
```

CONTAINER ID	IMAGE	STATUS	PORTS	COMMAND	CREATED	NAME
6b3f1ee5b464	mysql:5.7	Up About a minute	3306/tcp, 33060/tcp	"docker-entrypoint.s..."	About a minute ago	database_server
632b654c8cd5	ubuntu:20.04	Up About a minute		"tail -f /dev/null"	About a minute ago	monitoring_server
fc6987ddd75c	vulhub/struts2:2.3.30	Up 57 minutes	0.0.0.0:8080→8080/tcp, [::]:8080→8080/tcp	"/usr/local/bin/mvn-..."	57 minutes ago	s2-045-struts2-1

Figure 34: All three containers are running: database_server, monitoring_server and the Struts web server.

```
(kali@kali)-[~/security_governance_lab]
$ docker logs database_server --tail 30
```

```
2026-10-07T18:02:07.291004Z 0 [Note] InnoDB: Using Linux native AIO
2026-10-07T18:02:07.291477Z 0 [Note] InnoDB: Number of pools: 1
2026-10-07T18:02:07.291700Z 0 [Note] InnoDB: Using CPU crc32 instructions
2026-10-07T18:02:07.295414Z 0 [Note] InnoDB: Initializing buffer pool, total size = 128M, instances = 1, chunk size = 128M
2026-10-07T18:02:07.310190Z 0 [Note] InnoDB: Completed initialization of buffer pool
2026-10-07T18:02:07.315798Z 0 [Note] InnoDB: If the mysqld execution user is authorized, page cleaner thread priority can be changed. See the man page of setpriority().
2026-10-07T18:02:07.334251Z 0 [Note] InnoDB: Highest supported file format is Barracuda.
2026-10-07T18:02:07.366057Z 0 [Note] InnoDB: Creating shared tablespace for temporary tables
2026-10-07T18:02:07.367023Z 0 [Note] InnoDB: Setting file './ibtmp1' size to 12 MB. Physically writing the file full; Please wait ...
2026-10-07T18:02:07.403362Z 0 [Note] InnoDB: File './ibtmp1' size is now 12 MB.
2026-10-07T18:02:07.404935Z 0 [Note] InnoDB: 96 redo rollback segment(s) found. 96 redo rollback segment(s) are active.
2026-10-07T18:02:07.405002Z 0 [Note] InnoDB: 32 non-redo rollback segment(s) are active.
```

Figure 35: The first part of the database log: MySQL starting up.

```
2026-10-07T18:02:07.406875Z 0 [Note] Plugin 'FEDERATED' is disabled.
2026-10-07T18:02:07.411422Z 0 [Note] InnoDB: Buffer pool(s) load completed at 20261007 18:02:07
2026-10-07T18:02:07.416422Z 0 [Note] Found ca.pem, server-cert.pem and server-key.pem in data directory. Trying to enable SSL support using them.
2026-10-07T18:02:07.416483Z 0 [Note] Skipping generation of SSL certificates as certificate files are present in data directory.
2026-10-07T18:02:07.416491Z 0 [Warning] A deprecated TLS version TLSv1 is enabled. Please use TLSv1.2 or higher.
2026-10-07T18:02:07.416494Z 0 [Warning] A deprecated TLS version TLSv1.1 is enabled. Please use TLSv1.2 or higher.
2026-10-07T18:02:07.417529Z 0 [Warning] CA certificate ca.pem is self signed.
2026-10-07T18:02:07.417631Z 0 [Note] Skipping generation of RSA key pair as key files are present in data directory.
2026-10-07T18:02:07.418267Z 0 [Note] Server hostname (bind-address): '*'; port: 3306
2026-10-07T18:02:07.418514Z 0 [Note] IPv6 is available.
2026-10-07T18:02:07.419752Z 0 [Note] - '::' resolves to '::';
2026-10-07T18:02:07.419879Z 0 [Note] Server socket created on IP: '::'.
2026-10-07T18:02:07.422916Z 0 [Warning] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
2026-10-07T18:02:07.441060Z 0 [Note] Event Scheduler: Loaded 0 events
2026-10-07T18:02:07.441762Z 0 [Note] mysqld: ready for connections.
Version: '5.7.44' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server (GPL)
```

Figure 36: More of the database log, including warnings about an outdated encryption setting.

```
(kali@kali)-[~/security_governance_lab]
$ docker exec database_server mysql -uapp_user -papp_password -e "USE customer_data; SHOW TABLES;"
mysql: [Warning] Using a password on the command line interface can be insecure.
Tables_in_customer_data
customers

(kali@kali)-[~/security_governance_lab]
$ docker network inspect secgov_network
[
  {
    "Name": "secgov_network",
    "Id": "f3ab237558c1e78293f4104cc0e0eb77db3ef3345a3d1b77b082f5a53ef9ade9",
    "Created": "2026-10-07T13:37:35.245805816-04:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    }
  }
]
```

Figure 37: Logging in as `app_user` shows the `customers` table. The network inspection begins: subnet 172.19.0.0/16.

```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
": {
  "Name": "monitoring_server",
  "EndpointID": "1b82328c0f744a733148fc297b9e662a1312f1950f8a04672a9aaecee7b1773a",
  "MacAddress": "0a:6b:8c:ac:0b:ed",
  "IPv4Address": "172.19.0.3/16",
  "IPv6Address": ""
},
"6b3f1ee5b464f84a104ef8b7586dfbe2c0b49e5a870f27dbd8b87353b4c3a53d": {
  "Name": "database_server",
  "EndpointID": "1189d1c86b04c9dafff9da1ead8b57446cf7c509903f5140904d5ae7984bbec7",
  "MacAddress": "d2:b5:4c:3d:6d:52",
  "IPv4Address": "172.19.0.4/16",
  "IPv6Address": ""
},
"fc6987ddd75c371e9e063d73157d92617734b490f63d6840cd2e48f7e9f2039467f9feef4548578c4f": {
  "Name": "s2-045-struts2-1",
  "EndpointID": "5df0c7eea3c32f15d4ac0efb094714302074678a0b717c",
  "MacAddress": "3e:64:14:af:b4:81",
  "IPv4Address": "172.19.0.2/16",
  "IPv6Address": ""
},
"Options": {},
}
```

Figure 38: The network inspection continues: `monitoring_server` is at 172.19.0.3 and `database_server` is at 172.19.0.4.

```
(kali@kali)-[~/security_governance_lab]
$ find ~/security_governance_lab -maxdepth 2 -type f
/home/kali/security_governance_lab/database_init/init.sql
/home/kali/security_governance_lab/vulhub/README.md
/home/kali/security_governance_lab/vulhub/AGENTS.md
/home/kali/security_governance_lab/vulhub/.gitattributes
/home/kali/security_governance_lab/vulhub/.gitmodules
/home/kali/security_governance_lab/vulhub/.gitignore
/home/kali/security_governance_lab/vulhub/.autocorrectrc
/home/kali/security_governance_lab/vulhub/README.zh-cn.md
/home/kali/security_governance_lab/vulhub/LICENSE
/home/kali/security_governance_lab/vulhub/CONTRIBUTING.md
/home/kali/security_governance_lab/vulhub/environments.toml
/home/kali/security_governance_lab/docker-compose.yml
```

Figure 39: A listing of the files in the lab folder, including the `vulhub` project and `database_init/init.sql`.

4.7 Building the vulnerability scanner

The scanner is a Python program that checks the lab for three problems: the Struts flaw, default database credentials and weak network separation. It prints a summary and saves a report file. I created it in the nano editor, checked its syntax, made it executable and ran it.

How the three checks work

Check	Severity	What the scanner does
Apache Struts S2-045	Critical	Sends a web request to the web server on port 8080. A normal "success" reply (status 200) is reported as "Potentially Vulnerable" so a person can check by hand. A failed connection is reported as "Error".
MySQL default credentials	High	Tries to log in to the database as the administrator using the default password. If the login works it reports "Vulnerable". If the login is refused it reports "Not Vulnerable".
Network segmentation	Medium	Tries to reach the database port from the web server side using a small command called wget. If a connection is possible it reports "Vulnerable", because the two servers are not separated.

What the severity words mean. "Critical" means a serious flaw that attackers can use quickly and that can cause great harm, so it must be repaired first. "High" means a serious weakness that should be fixed soon. "Medium" means a weakness that makes other attacks easier or more damaging. A governance process should set a time limit for each level. In this lab the target is seven days.

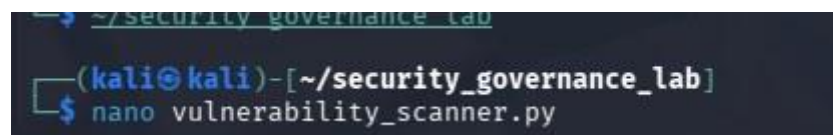
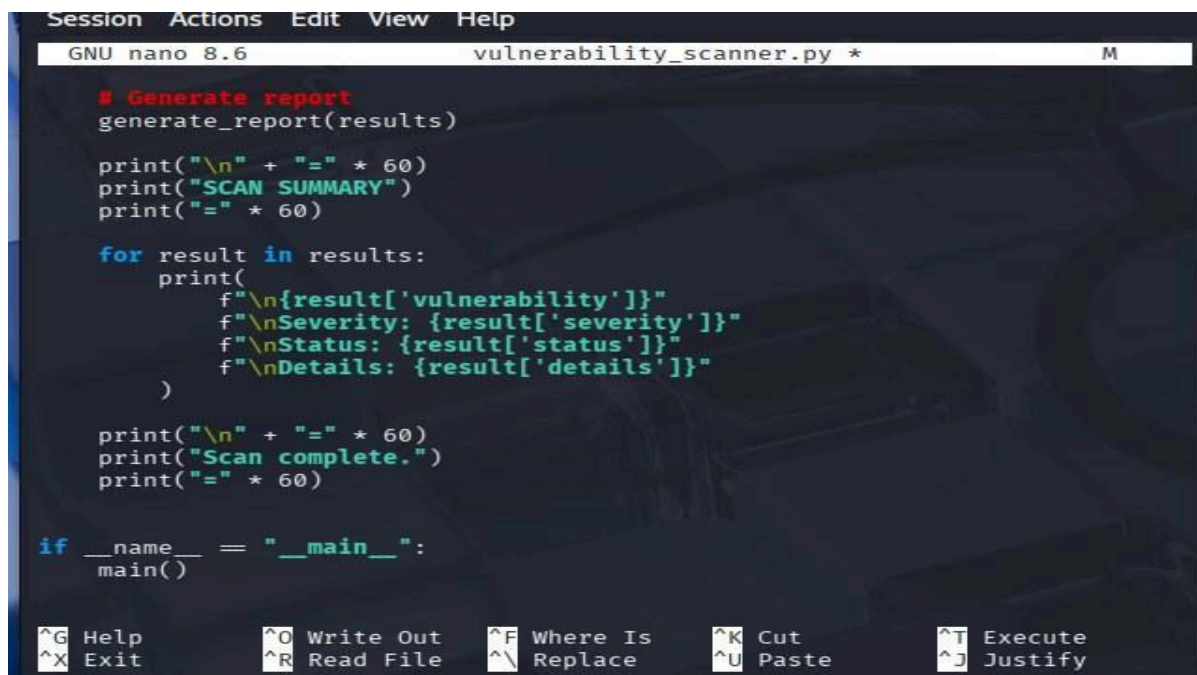


Figure 40: Opening vulnerability_scanner.py in the nano editor.



Figure 41: The top of the scanner program: the import lines.



```
Session Actions Edit View Help
GNU nano 8.6 vulnerability_scanner.py * M

# Generate report
generate_report(results)

print("\n" + "=" * 60)
print("SCAN SUMMARY")
print("=" * 60)

for result in results:
    print(
        f"\n{result['vulnerability']}"
        f"\nSeverity: {result['severity']}"
        f"\nStatus: {result['status']}"
        f"\nDetails: {result['details']}"
    )

print("\n" + "=" * 60)
print("Scan complete.")
print("=" * 60)

if __name__ == "__main__":
    main()
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File Replace Paste Justify

Figure 42: The end of the scanner program: the summary printout and the main section.



```
(kali@kali)-[~/security_governance_lab]
$ nano vulnerability_scanner.py

(kali@kali)-[~/security_governance_lab]
$ ls -l vulnerability_scanner.py
-rw-rw-r-- 1 kali kali 6998 Oct  7 14:27 vulnerability_scanner.py
```

Figure 43: The scanner file is saved (6,998 bytes).

First scan: two checks fail with errors

The first run produced a report (**vulnerability_report_20261007_143035.json**), but two of the three checks failed because of problems with my set-up, not because the lab was safe:

- **Struts check:** the scanner could not find the name **s2-045-struts2-1**. Docker only lets containers find each other by name inside the Docker network. My scanner runs on the Kali machine itself, outside that network.
- **Network check:** the test sends a request to the database port, and MySQL replies with raw binary data. Python tried to read it as normal text and stopped with a decoding error.
- **Database check:** this one worked. It reported **Vulnerable**, because the default root password could be used.

```
(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile vulnerability_scanner.py

(kali@kali)-[~/security_governance_lab]
$ chmod +x vulnerability_scanner.py

(kali@kali)-[~/security_governance_lab]
$ ./vulnerability_scanner.py

=====
Security Governance Vulnerability Scanner
=====

[*] Checking Apache Struts S2-045 vulnerability ...
[*] Checking MySQL default credentials ...
[*] Checking network segmentation ...

[+] Report generated: vulnerability_report_20261007_143035.json

=====
SCAN SUMMARY
=====

Apache Struts S2-045
Severity: Critical
Status: Error
Details: Connection error: HTTPConnectionPool(host='s2-045-struts2-1', port=8080): Max retries exceeded with url: / (Caused by NameResolutionError("<urlli
```

Figure 44: Syntax check, making the file executable and the first scan. The Struts check shows an error.

```
SCAN SUMMARY

Apache Struts S2-045
Severity: Critical
Status: Error
Details: Connection error: HTTPConnectionPool(host='s2-045-struts2-1', port=8080): Max retries exceeded with url: / (Caused by NameResolutionError("<urllib3.connection.HTTPConnection object at 0x7f68ce7f9e80>: Failed to resolve 's2-045-struts2-1' ([Errno -2] Name or service not known)"))

MySQL Default Credentials
Severity: High
Status: Vulnerable
Details: MySQL root account is accessible using the default password.

Insufficient Network Segmentation
Severity: Medium
Status: Error
Details: Error checking network segmentation: 'utf-8' codec can't decode byte 0xff in position 25: invalid start byte

=====
Scan complete.
=====
```

Figure 45: First scan results: Struts check shows an error (name not found), the database is Vulnerable and the network check shows an error (decoding problem).

Fix 1: use the container's network address

I inspected the network to find the real addresses (Struts 172.19.0.2, monitoring 172.19.0.3 and database 172.19.0.4) and changed the scanner to use the address instead of the name.

```
(kali@kali)-[~/security_governance_lab]
$ docker network inspect secgov_network
[
  {
    "Name": "secgov_network",
    "Id": "f3ab237558c1e78293f4104cc0e0eb77db3ef3345a3d1b77b082f5a53ef9ade9",
    "Created": "2026-10-07T13:37:35.245805816-04:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    }
  }
]
```

Figure 46: Inspecting secgov_network to find the container addresses.

```
Session Actions Edit View Help
"ConfigOnly": false,
"Containers": {
  "632b654c8cd517b54dbf17c67de00da1beba5b99fb7310e703c8fc8e43300903": {
    "Name": "monitoring_server",
    "EndpointID": "1b82328c0f744a733148fc297b9e662a1312f1950f8a04672a9aaecee7b1773a",
    "MacAddress": "0a:6b:8c:ac:0b:ed",
    "IPv4Address": "172.19.0.3/16",
    "IPv6Address": ""
  },
  "6b3f1ee5b464f84a104ef8b7586dfbe2c0b49e5a870f27dbd8b87353b4c3a53d": {
    "Name": "database_server",
    "EndpointID": "1189d1c86b04c9da99f9da1ead8b57446cf7c509903f5140904d5ae7984bbec7",
    "MacAddress": "d2:b5:4c:3d:6d:52",
    "IPv4Address": "172.19.0.4/16",
    "IPv6Address": ""
  },
  "fc6987ddd75c371e9e063d73157d92617734b490f63d6840cd2e48f7e9f20394": {
    "Name": "s2-045-struts2-1",
    "EndpointID": "5df0c7eea3c32f15d4ac0efb094714302074678a0b717c67f9feef4548578c4f",
    "MacAddress": "3e:64:14:af:b4:81",
    "IPv4Address": "172.19.0.2/16",
    "IPv6Address": ""
  }
}
```

Figure 47: The rest of the inspection output, showing each container and its address.

```
# Configuration
STRUTS_CONTAINER = "s2-045-struts2-1"
DATABASE_CONTAINER = "database_server"
STRUTS_URL = "http://172.19.0.2:8080"
```

Figure 48: The scanner settings now use the address http://172.19.0.2:8080 for the Struts check.

```
(kali@kali)-[~/security_governance_lab]
$ ./vulnerability_scanner.py

Security Governance Vulnerability Scanner

[*] Checking Apache Struts S2-045 vulnerability ...
[*] Checking MySQL default credentials ...
[*] Checking network segmentation ...

[+] Report generated: vulnerability_report_20261007_144614.json

SCAN SUMMARY

Apache Struts S2-045
Severity: Critical
Status: Potentially Vulnerable
Details: Server responded with HTTP status 200. Manual verification is recommended.

MySQL Default Credentials
Severity: High
Status: Vulnerable
Details: MySQL root account is accessible using the default password.
```

Figure 49: Second scan: Struts is now "Potentially Vulnerable" and the database is Vulnerable.

```
MySQL Default Credentials
Severity: High
Status: Vulnerable
Details: MySQL root account is accessible using the default password.

Insufficient Network Segmentation
Severity: Medium
Status: Error
Details: Error checking network segmentation: 'utf-8' codec can't decode byte
0xff in position 25: invalid start byte

Scan complete.
```

Figure 50: The rest of the second scan: the network check still shows a decoding error.

Fix 2: stop Python decoding binary data

I located the network check in the program and removed the option that made Python treat the answer as text. The command then collects the answer as raw data, which no longer causes an error.

```
(kali@kali)-[~/security_governance_lab]
$ grep -n "check_mysql\|results.append" vulnerability_scanner.py
77: def check_mysql_credentials():
208:     results.append(check_struts_vulnerability())
211:     results.append(check_mysql_credentials())
214:     results.append(check_network_segmentation())
```

Figure 51: Searching the scanner for the lines where the checks are called.

```

try:
    command = [
        "docker",
        "exec",
        STRUTS_CONTAINER,
        "wget",
        "-q",
        "-O",
        "-",
        "http://database_server:3306"
    ]

    process = subprocess.run(
        command,
        capture_output=True
    )

```

Figure 52: The network check in the editor: the text option is removed so binary data can be read safely.

Final scan result

After both fixes the scanner worked fully and saved **vulnerability_report_20261007_145614.json**.

```

[+] Report generated: vulnerability_report_20261007_145614.json
=====
SCAN SUMMARY
=====

Apache Struts S2-045
Severity: Critical
Status: Potentially Vulnerable
Details: Server responded with HTTP status 200. Manual verification is recommended.

MySQL Default Credentials
Severity: High
Status: Vulnerable
Details: MySQL root account is accessible using the default password.

Insufficient Network Segmentation
Severity: Medium
Status: Vulnerable
Details: Web server can directly reach the database server.

=====
Scan complete.
=====

```

Figure 53: The final scan: Struts Critical (Potentially Vulnerable), database High (Vulnerable) and network Medium (Vulnerable).

```

(kali@kali)-[~/security_governance_lab]
$ ls -lh vulnerability_report_20261007_145614.json
-rw-rw-r-- 1 kali kali 1.2K Oct  7 14:56 vulnerability_report_20261007_145614.json

(kali@kali)-[~/security_governance_lab]
$

```

Figure 54: The saved scan report file (1.2 kilobytes).

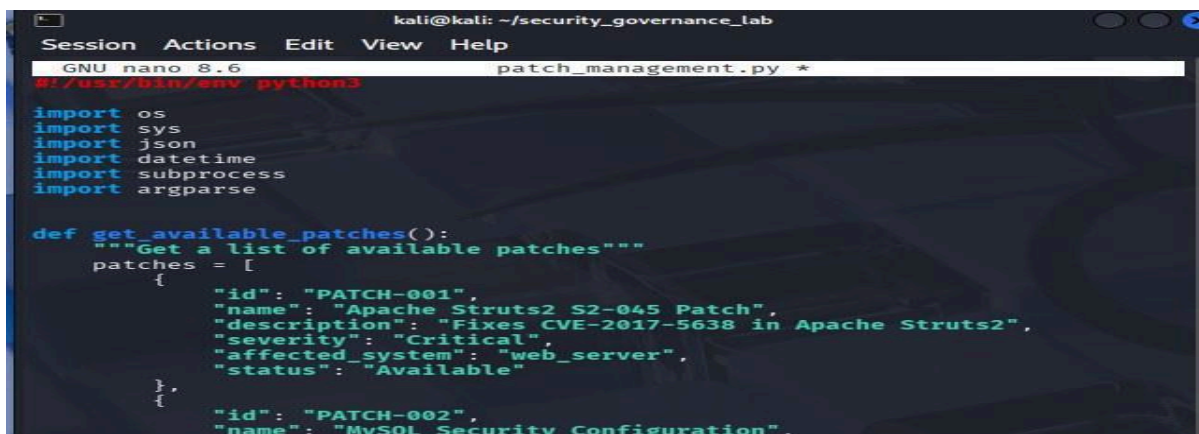
Finding	Severity	Status	What it means
Apache Struts S2-045	Critical	Potentially Vulnerable	The server replied with a normal "success" message (status 200) and the scanner asked for manual checking. The version in use (2.3.30) is known to be affected.

Finding	Severity	Status	What it means
MySQL default credentials	High	Vulnerable	The database administrator account could be opened with the default password.
Network segmentation	Medium	Vulnerable	The web server can reach the database server directly, with no barrier between them.

The scanner did not prove the Struts flaw by attacking it. It saw a normal reply and asked a person to verify. For that reason I describe the Struts result as "potentially vulnerable" and not as confirmed.

4.8 Building the patch management tool

The patch tool lists three available "patches" and can apply each one: **PATCH-001** (Struts, Critical), **PATCH-002** (database security, High) and **PATCH-003** (network segmentation, Medium). In this lab the tool only **simulates** patching. It prints messages and writes a small status file. It does not change the real containers.



```

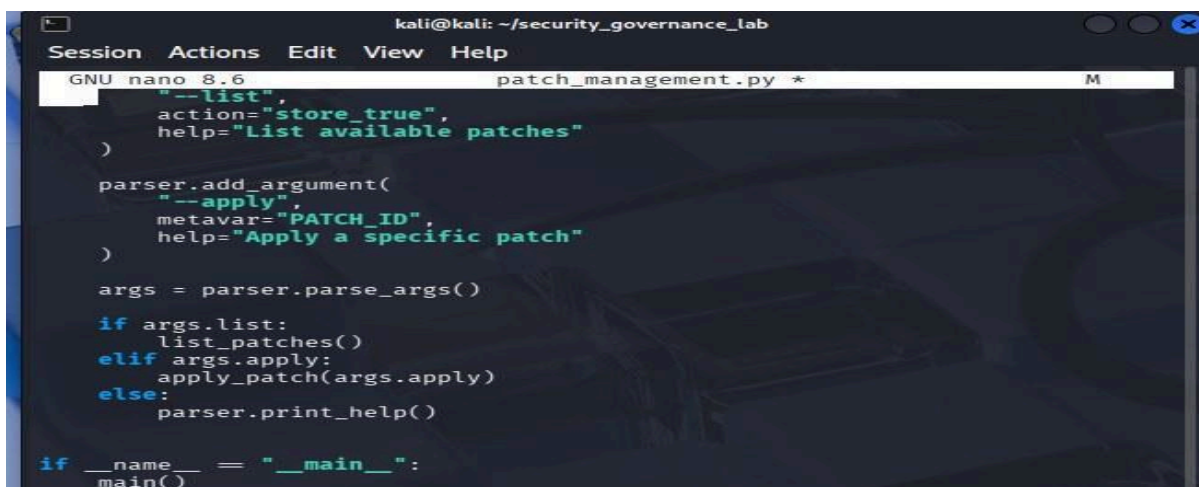
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 patch_management.py *
#!/usr/bin/env python3

import os
import sys
import json
import datetime
import subprocess
import argparse

def get_available_patches():
    """Get a list of available patches"""
    patches = [
        {
            "id": "PATCH-001",
            "name": "Apache Struts2 S2-045 Patch",
            "description": "Fixes CVE-2017-5638 in Apache Struts2",
            "severity": "Critical",
            "affected_system": "web_server",
            "status": "Available"
        },
        {
            "id": "PATCH-002",
            "name": "MySQL Security Configuration",

```

Figure 55: The patch tool in the nano editor: the list of three available patches.



```

kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 patch_management.py *
M
    "--list",
    action="store_true",
    help="List available patches"
)

parser.add_argument(
    "--apply",
    metavar="PATCH_ID",
    help="Apply a specific patch"
)

args = parser.parse_args()

if args.list:
    list_patches()
elif args.apply:
    apply_patch(args.apply)
else:
    parser.print_help()

if __name__ == "__main__":
    main()

```

Figure 56: The end of the patch tool: the options --list and --apply.

```
(kali@kali)-[~/security_governance_lab]
$ nano patch_management.py

(kali@kali)-[~/security_governance_lab]
$ ls -lh patch_management.py
-rw-rw-r-- 1 kali kali 3.5K Oct  8 04:03 patch_management.py

(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile patch_management.py

(kali@kali)-[~/security_governance_lab]
$
```

Figure 57: The file is saved (3.5 kilobytes) and its syntax is checked.

```
(kali@kali)-[~/security_governance_lab]
$ python3 patch_management.py --list
Available patches:

ID: PATCH-001
Name: Apache Struts2 S2-045 Patch
Description: Fixes CVE-2017-5638 in Apache Struts2
Severity: Critical
Affected System: web_server
Status: Available

ID: PATCH-002
Name: MySQL Security Configuration
Description: Secures MySQL with proper authentication and access controls
Severity: High
Affected System: database_server
Status: Available

ID: PATCH-003
Name: Network Segmentation Rules
Description: Implements proper network segmentation between servers
Severity: Medium
Affected System: all
Status: Available
```

Figure 58: Listing the three available patches with their severity and affected system.

```
(kali@kali)-[~/security_governance_lab]
$ python3 patch_management.py --apply PATCH-001
Applying patch: Apache Struts2 S2-045 Patch (PATCH-001)
Description: Fixes CVE-2017-5638 in Apache Struts2
Affected system: web_server
Simulating Struts patch application...
Struts patch applied successfully

(kali@kali)-[~/security_governance_lab]
$ python3 patch_management.py --apply PATCH-002
Applying patch: MySQL Security Configuration (PATCH-002)
Description: Secures MySQL with proper authentication and access controls
Affected system: database_server
Simulating MySQL security configuration...
MySQL security configuration applied successfully

(kali@kali)-[~/security_governance_lab]
$ python3 patch_management.py --apply PATCH-003
Applying patch: Network Segmentation Rules (PATCH-003)
Description: Implements proper network segmentation between servers
Affected system: all
Simulating network segmentation implementation...
Network segmentation rules applied successfully
```

Figure 59: Applying PATCH-001, PATCH-002 and PATCH-003 one after the other. Each reports success.

```
(kali@kali)-[~/security_governance_lab]
$ cat patch_status.json
{"network_segmented": true}
```

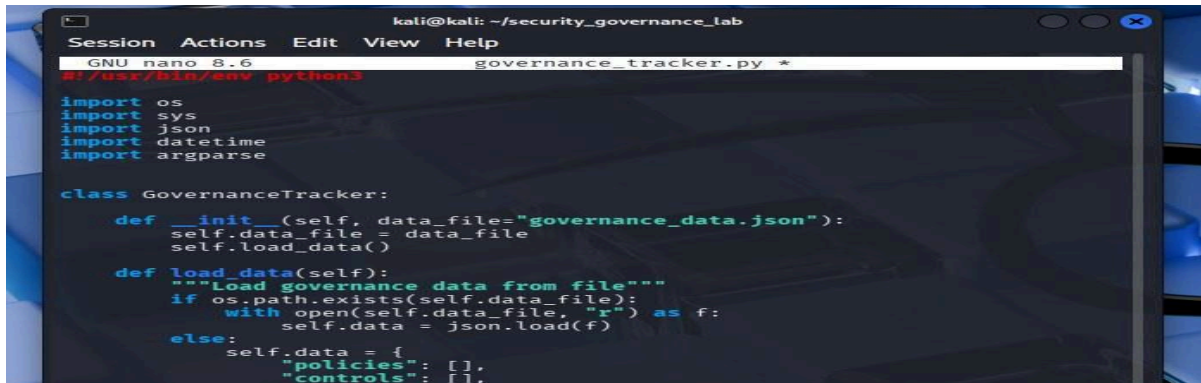
Figure 60: The status file contains only {"network_segmented": true}.

A weakness in the patch tool that I found

The last screenshot shows a problem. The tool **overwrites** the status file every time it applies a patch, so only the most recent patch is remembered. Here the last patch was PATCH-003, so only "network_segmented" appears. This explains confusing results later in Part 3.

4.9 Building the governance tracker

The governance tracker is a record-keeping program. It stores **policies** (rules), **controls** (things that carry out the rules), **risks**, **incidents**, **metrics** and **audits**. It can show a summary dashboard. A second script fills it with sample data, such as a Patch Management Policy, an Incident Response Policy and a recorded web server compromise.



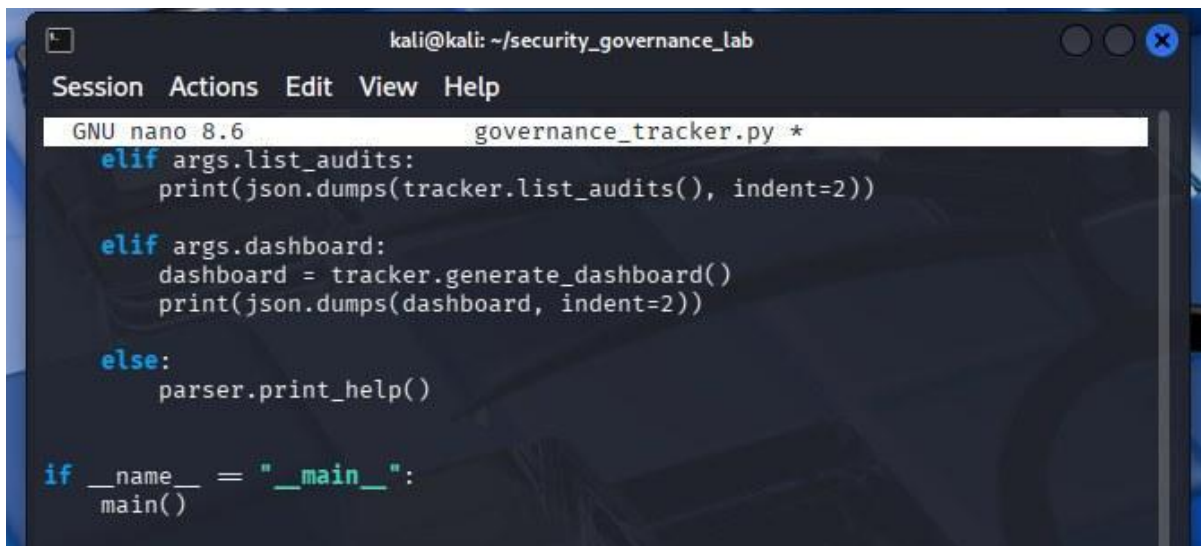
```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 governance_tracker.py *
#!/usr/bin/env python3

import os
import sys
import json
import datetime
import argparse

class GovernanceTracker:
    def __init__(self, data_file="governance_data.json"):
        self.data_file = data_file
        self.load_data()

    def load_data(self):
        """Load governance data from file"""
        if os.path.exists(self.data_file):
            with open(self.data_file, "r") as f:
                self.data = json.load(f)
        else:
            self.data = {
                "policies": [],
                "controls": []
            }
```

Figure 61: The governance tracker program in the nano editor: the GovernanceTracker class.



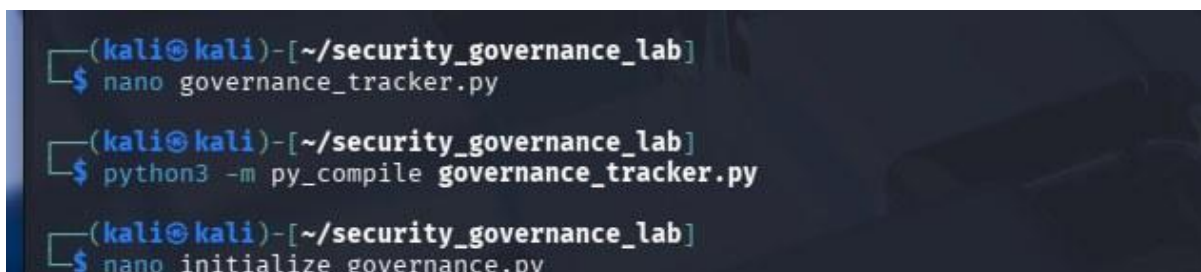
```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 governance_tracker.py *
elif args.list_audits:
    print(json.dumps(tracker.list_audits(), indent=2))

elif args.dashboard:
    dashboard = tracker.generate_dashboard()
    print(json.dumps(dashboard, indent=2))

else:
    parser.print_help()

if __name__ == "__main__":
    main()
```

Figure 62: The end of the tracker: the dashboard option and the main section.



```
(kali@kali)-[~/security_governance_lab]
$ nano governance_tracker.py

(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile governance_tracker.py

(kali@kali)-[~/security_governance_lab]
$ nano initialize_governance.py
```

Figure 63: Creating and checking the tracker, then starting the initialisation script.

```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 initialize_governance.py *
#!/usr/bin/env python3
import os
import sys
import json
import datetime
from governance_tracker import GovernanceTracker

def initialize_governance_data():
    """Initialize the governance tracker with sample data"""
    tracker = GovernanceTracker()

    # Add policies
    patch_policy_id = tracker.add_policy(
        "Patch Management Policy",
        "Policy governing the timely application of security patches",
        "CISO",
        "2023-01-15",
        "2024-01-15"
    )
    vuln_policy_id = tracker.add_policy(
        "Vulnerability Management Policy",
        "Policy governing the identification and remediation of security vul",
        "Security Director",
        "2023-02-10",
```

Figure 64: The initialisation script adds the first policies.

```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 initialize_governance.py *
    tracker.add_metric(
        "Security Incidents",
        "Number of security incidents in the period",
        0.0,
        1.0,
        "May 2023",
        "Stable"
    )

    # Add audits
    tracker.add_audit(
        "Annual Security Audit",
        "Comprehensive audit of security controls",
        "2023-05-01",
        "External Auditor",
        "Several findings related to patch management and network segmentati",
        "Improve patch management process and implement network segmentation"
    )

    print("Governance tracker initialized with sample data")

if __name__ == "__main__":
    initialize_governance_data()
```

Figure 65: The end of the initialisation script: a metric, an audit and the final message.

```
(kali@kali)-[~/security_governance_lab]
$ nano governance_tracker.py

(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile governance_tracker.py

(kali@kali)-[~/security_governance_lab]
$ nano initialize_governance.py

(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile initialize_governance.py

(kali@kali)-[~/security_governance_lab]
$ chmod +x initialize_governance.py

(kali@kali)-[~/security_governance_lab]
$ ./initialize_governance.py
Governance tracker initialized with sample data

(kali@kali)-[~/security_governance_lab]
$ python3 governance_tracker.py --dashboard
```

Figure 66: Running the initialisation script. It confirms: "Governance tracker initialized with sample data".

```

kali@kali: ~/security_governance_lab
Session Actions Edit View Help

(kali@kali)-[~/security_governance_lab]
$ python3 governance_tracker.py --dashboard
{
  "generated_date": "2026-10-08T04:24:36.170583",
  "summary": {
    "policies": 3,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 3,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {

```

Figure 67: The tracker dashboard output: 3 policies, 3 controls, 3 risks, 1 incident, 3 metrics and 1 audit, with 1 critical, 1 high and 1 medium risk.

```

    },
    {
      "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
    },
    {
      "name": "Vulnerability Remediation Time",
      "target": 7.0,
      "actual": 12.0,
      "status": "Green"
    },
    {
      "name": "Security Incidents",
      "target": 0.0,
      "actual": 1.0,
      "status": "Green"
    }
  ]
}
(kali@kali)-[~/security_governance_lab]
$

```

Figure 68: The metric status lines. Patch Compliance is Red (85 against a target of 95).

```

(kali@kali)-[~/security_governance_lab]
$ ls -lh governance_data.json
-rw-rw-r-- 1 kali kali 4.6K Oct  8 04:23 governance_data.json

```

Figure 69: The saved tracker data file (4.6 kilobytes).

Metric	Target	Actual	Status shown by the tracker
Patch Compliance (percentage)	95	85	Red (correct: below target)
Vulnerability Remediation Time (days)	7	12	Green (wrong: 12 days is slower than the 7-day target)
Security Incidents (number)	0	1	Green (wrong: one incident is worse than zero)

A second weakness that I found

The tracker marks a metric Green whenever the actual value is greater than or equal to the target. That works for "higher is better" numbers such as patch compliance. It gives the wrong answer for "lower is better" numbers such as remediation time and incidents. A dashboard that shows false Green lights is dangerous, because leaders may feel safe when they should not. A better design would store, for each metric, whether higher or lower is better.

4.10 How this environment reflects the Equifax breach

The lab copies the main conditions that led to the Equifax breach. First, the public web server runs Struts 2.3.30, which has a known and serious flaw (CVE-2017-5638), even though a fix has long been available. This is the same weakness that was used against Equifax. Second, the database holds sensitive customer details and is protected by a default password. Third, the web server can talk directly to the database, so anyone who takes over the web server can reach the data. At Equifax, attackers also moved from system to system because the network was not split into protected zones.

The lab also copies the governance side of the story. Policies for patching and vulnerability management exist in the tracker, yet the critical flaw is still open, patch compliance is only 85 per cent against a target of 95, and fixing critical problems takes 12 days instead of 7. There is no live monitoring to raise an alarm. In other words, the rules exist on paper but the organisation is not following or checking them. This gap between written policy and daily practice is what governance is meant to close, and it was the real root cause at Equifax.

5. Part 2: Analysing security governance failures

5.1 The attack simulation

The attack script checks whether the Struts server is vulnerable and, if so, **pretends** to take data from the database. It prints messages such as "Simulating exfiltration of customer data" and writes a small log of what happened. It does not really steal anything. Its purpose is to give us a record to analyse.

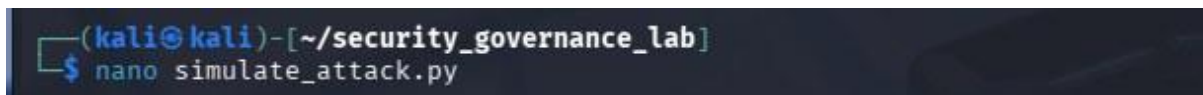
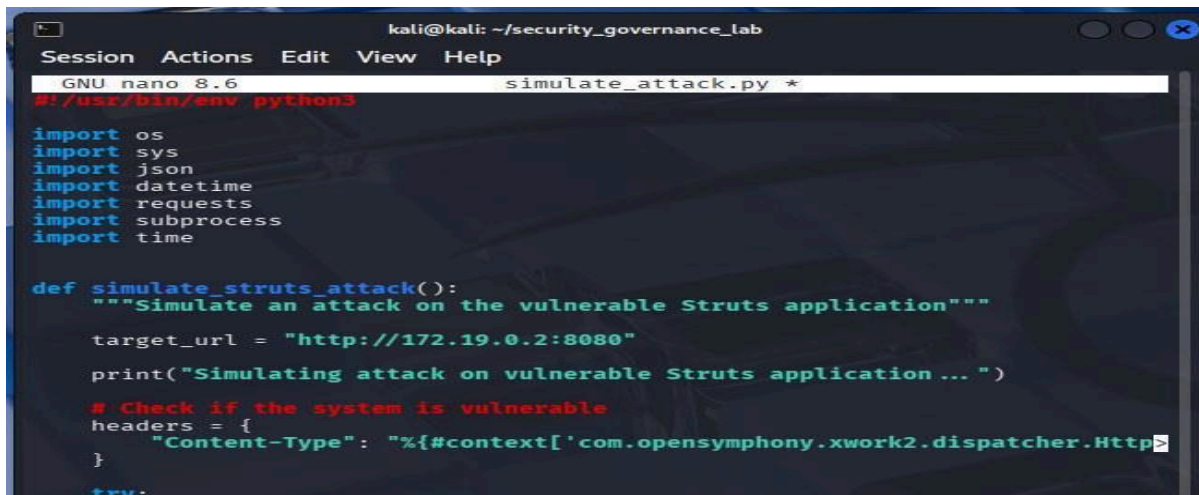


Figure 70: Opening `simulate_attack.py` in the nano editor.



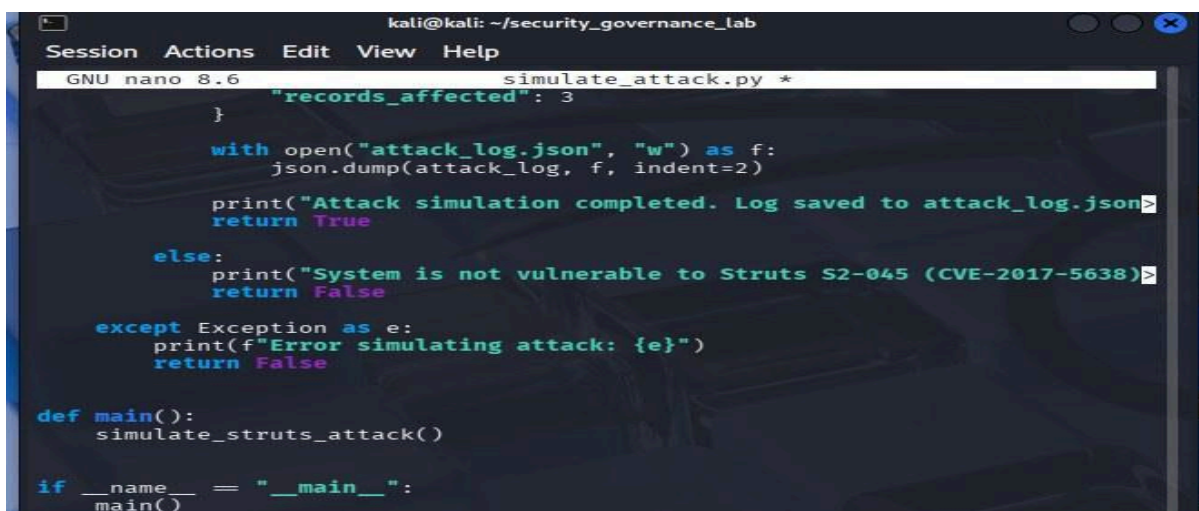
```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 simulate_attack.py *
#!/usr/bin/env python3

import os
import sys
import json
import datetime
import requests
import subprocess
import time

def simulate_struts_attack():
    """Simulate an attack on the vulnerable Struts application"""
    target_url = "http://172.19.0.2:8080"
    print("Simulating attack on vulnerable Struts application ... ")
    # Check if the system is vulnerable
    headers = {
        "Content-Type": "%{#context['com.opensymphony.xwork2.dispatcher.Http>
    }

    try:
```

Figure 71: The top of the attack script. The first version aims at `http://172.19.0.2:8080`.



```
        "records_affected": 3
    }

    with open("attack_log.json", "w") as f:
        json.dump(attack_log, f, indent=2)

    print("Attack simulation completed. Log saved to attack_log.json")
    return True

    else:
        print("System is not vulnerable to Struts S2-045 (CVE-2017-5638)")
        return False

    except Exception as e:
        print(f"Error simulating attack: {e}")
        return False

def main():
    simulate_struts_attack()

if __name__ == "__main__":
    main()
```

Figure 72: The end of the attack script: it writes the attack log and handles errors.

Problem: the first runs could not connect

The first runs stopped with "Connection refused" for the address 172.19.0.2 on port 8080. I confirmed that all three containers were running, and I tested the web server from the terminal with `curl http://localhost:8080`. That worked and returned the Struts test page. So the server was healthy, but the container's internal address was not reliably reachable from the Kali machine. The port published on **localhost** was reachable. I did not investigate the exact cause further, and the screenshots do not prove why the internal address failed.

```

(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile simulate_attack.py

(kali@kali)-[~/security_governance_lab]
$ ./simulate_attack.py
zsh: permission denied: ./simulate_attack.py

(kali@kali)-[~/security_governance_lab]
$ chmod +x simulate_attack.py
chmod: cannot access 'simulate_attack.py': No such file or directory

(kali@kali)-[~/security_governance_lab]
$ cd ~/security_governance_lab

(kali@kali)-[~/security_governance_lab]
$ chmod +x simulate_attack.py

(kali@kali)-[~/security_governance_lab]
$ ./simulate_attack.py
Simulating attack on vulnerable Struts application ...
Error simulating attack: HTTPConnectionPool(host='172.19.0.2', port=8080): Max
x retries exceeded with url: / (Caused by NewConnectionError('<urllib3.connec
tion.HTTPConnection object at 0x7fd949901d30>: Failed to establish a new conn
ection: [Errno 111] Connection refused'))

```

Figure 73: Checking the script, fixing a permission problem and a typing mistake, and the first run, which fails to connect.

```

(kali@kali)-[~/security_governance_lab]
$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED
6b3f1ee5b464	mysql:5.7	"docker-entrypoint.s..."	15 hours ago
Up 46 minutes	3306/tcp, 33060/tcp		database_serv
632b654c8cd5	ubuntu:20.04	"tail -f /dev/null"	15 hours ago
Up 46 minutes			monitoring_se
fc6987ddd75c	vulhub/struts2:2.3.30	"/usr/local/bin/mvn-..."	16 hours ago
Up 46 minutes	0.0.0.0:8080→8080/tcp, [::]:8080→8080/tcp		s2-045-struts

```

2-1

```

Figure 74: docker ps: all three containers are running.

```

(kali@kali)-[~/security_governance_lab]
$ curl http://localhost:8080

```

```

<html>
<head>
  <title>Struts2 Showcase - Fileupload sample</title>
</head>
<body>
<div class="page-header">
  <h1>Fileupload sample</h1>
</div>
<div class="container-fluid">
  <div class="row-fluid">
    <div class="span12">
      <form id="doUpload" name="doUpload" action="/doUpload.action;jsessio
nid=1mpc4uv459fl0hlvm0rphd9ys" method="POST" enctype="multipart/form-data">
<table class="wwFormTable">
  <tr>
    <td class="tdLabel"><label for="doUpload_upload" class="label">File:</lab
el></td>
    <td>
      <input type="file" name="upload" value="" id="doUpload_upload"/></td>
  </tr>

```

Figure 75: curl http://localhost:8080 returns the Struts "Fileupload sample" page. The server is working.

```
<tr>
  <td class="tdLabel"><label for="doUpload_caption" class="label">Caption:<
/label></td>
  <td>
    <input type="text" name="caption" value="" id="doUpload_caption" /></td>
</tr>

<tr>
  <td colspan="2"><div align="right"><input type="submit" value="Submit" id
="doUpload_0" class="btn btn-primary" />
</div></td>
</tr>

</table></form>

</div>
</div>
</div>
</body>
</html>

(kali@kali)-[~/security_governance_lab]
```

Figure 76: The rest of the page returned by curl.

```
(kali@kali)-[~/security_governance_lab]
$ ./simulate_attack.py
simulating attack on vulnerable Struts application ...
error simulating attack: HTTPConnectionPool(host='172.19.0.2', port=8080): Ma
retries exceeded with url: / (Caused by NewConnectionError('<urllib3.connec
ion.HTTPConnection object at 0x7f8c982d1d30>: Failed to establish a new conn
ction: [Errno 111] Connection refused'))

(kali@kali)-[~/security_governance_lab]
$ ./simulate_attack.py
simulating attack on vulnerable Struts application ...
error simulating attack: HTTPConnectionPool(host='172.19.0.2', port=8080): Ma
retries exceeded with url: / (Caused by NewConnectionError('<urllib3.connec
ion.HTTPConnection object at 0x7f7e95a15d30>: Failed to establish a new conn
ction: [Errno 111] Connection refused'))
```

Figure 77: The attack script fails twice with "Connection refused" for 172.19.0.2:8080.

Fix: aim the script at localhost

```
(kali@kali)-[~/security_governance_lab]
$ nano simulate_attack.py
```

Figure 78: Opening simulate_attack.py again to change the target address.

```
def simulate_struts_attack():
    """Simulate an attack on the vulnerable Struts application"""

    target_url = "http://localhost:8080"

    print("Simulating attack on vulnerable Struts application ...")
```

Figure 79: The target is changed to http://localhost:8080.

After the change, the simulation ran to the end. It reported that the system was vulnerable to Struts S2-045 (CVE-2017-5638), then simulated database access and the taking of customer data, and finally saved the log **attack_log.json** (209 bytes, three customer records affected).

```
(kali@kali)-[~/security_governance_lab]
$ nano simulate_attack.py

(kali@kali)-[~/security_governance_lab]
$ ./simulate_attack.py
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json

(kali@kali)-[~/security_governance_lab]
$
```

Figure 80: The attack simulation completes and saves its log. Struts S2-045 is detected, and data theft is simulated.

5.2 Analysing the governance failures

The analysis script reads four sources: the vulnerability report, the attack log, the governance tracker data and the patch status file. It then lists governance failures. I opened it first in the editor to check it.

```
(kali@kali)-[~/security_governance_lab]
$ nano analyze_governance_failures.py
```

Figure 81: Opening `analyze_governance_failures.py`.

```

      kali@kali: ~/security_governance_lab
Session  Actions  Edit  View  Help
GNU nano 8.6      analyze_governance_failures.py *
#!/usr/bin/env python3

import os
import sys
import json
import datetime

def load_vulnerability_report():
    """Load the latest vulnerability report"""
    report_files = [
        f for f in os.listdir(".")
        if f.startswith("vulnerability_report_") and f.endswith(".json")
    ]

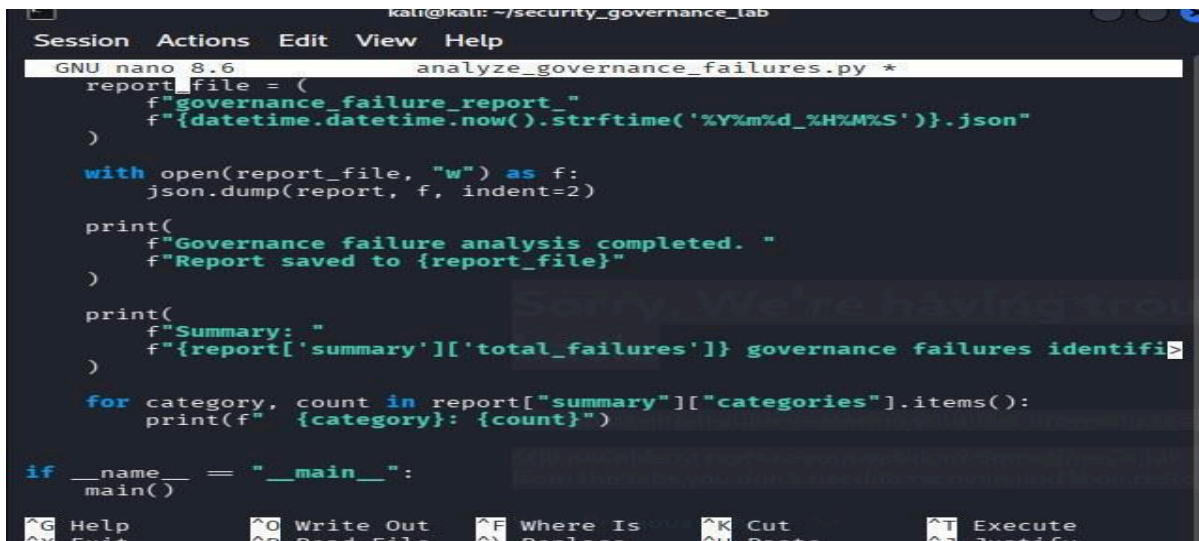
    if not report_files:
        print("No vulnerability reports found")
        return None

    latest_report = max(report_files)

    with open(latest_report, "r") as f:
        return json.load(f)

^G Help      ^O Write Out  ^F Where Is    ^K Cut        ^T Execute
^X Exit      ^R Read File   ^_ Replace     ^U Paste      ^I Justify
```

Figure 82: The top of the analysis script: it loads the latest vulnerability report.



```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 analyze_governance_failures.py *
report_file = (
    f"governance_failure_report."
    f"{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
)

with open(report_file, "w") as f:
    json.dump(report, f, indent=2)

print(
    f"Governance failure analysis completed. "
    f"Report saved to {report_file}"
)

print(
    f"Summary: "
    f"{report['summary']['total_failures']} governance failures identi>
)

for category, count in report["summary"]["categories"].items():
    print(f" {category}: {count}")

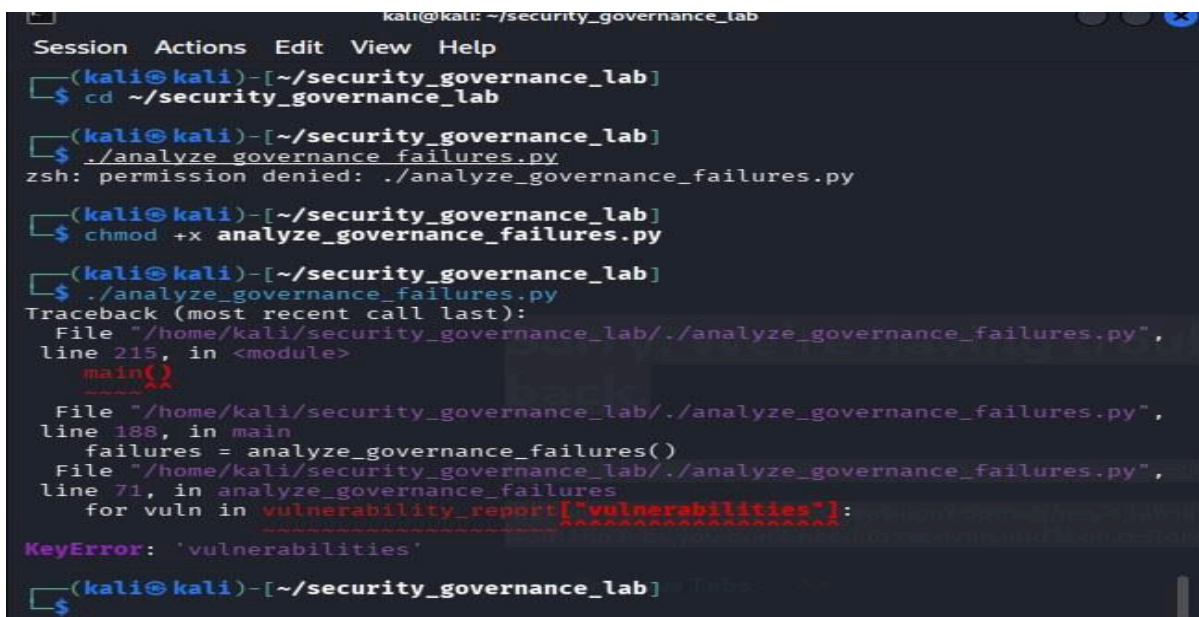
if __name__ == "__main__":
    main()

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^I Justify
```

Figure 83: The end of the analysis script: it writes the failure report and prints a summary.

Problem: the script did not match my report

The script stopped with an error. The cause was that the lab sheet's scanner writes its findings under the name **vulnerabilities**, while my scanner writes them under **results** and gives each one a **status** and **details** field. I printed my report to confirm this and then corrected the script in two places.



```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
(kali@kali)-[~/security_governance_lab]
$ cd ~/security_governance_lab
(kali@kali)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
zsh: permission denied: ./analyze_governance_failures.py
(kali@kali)-[~/security_governance_lab]
$ chmod +x analyze_governance_failures.py
(kali@kali)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
Traceback (most recent call last):
  File "/home/kali/security_governance_lab/./analyze_governance_failures.py",
    line 215, in <module>
    main()
  File "/home/kali/security_governance_lab/./analyze_governance_failures.py",
    line 188, in main
    failures = analyze_governance_failures()
  File "/home/kali/security_governance_lab/./analyze_governance_failures.py",
    line 71, in analyze_governance_failures
    for vuln in vulnerability_report["vulnerabilities"]:
KeyError: 'vulnerabilities'
(kali@kali)-[~/security_governance_lab]
```

Figure 84: First run of the analysis script: a permission problem is fixed, and the run then stops with an error.

```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help

(kali@kali)~[~/security_governance_lab]
$ cat vulnerability_report_20261007_145614.json
{
  "scan_date": "2026-10-07 14:56:14",
  "scanner": "Security Governance Vulnerability Scanner",
  "results": [
    {
      "vulnerability": "Apache Struts S2-045",
      "cve": "CVE-2017-5638",
      "severity": "Critical",
      "status": "Potentially Vulnerable",
      "details": "Server responded with HTTP status 200. Manual verification is recommended.",
      "remediation": "Upgrade Apache Struts to version 2.3.32 or 2.5.10.1 or later"
    },
    {
      "vulnerability": "MySQL Default Credentials",
      "severity": "High",
      "status": "Vulnerable",
      "details": "MySQL root account is accessible using the default password.",
      "remediation": "Change default/root credentials and use strong passwords"
    },
    {
      "vulnerability": "Insufficient Network Segmentation",

```

Figure 85: My vulnerability report printed in full. The findings are stored under "results".

```
      "status": "Vulnerable",
      "details": "MySQL root account is accessible using the default password.",
      "remediation": "Change default/root credentials and use strong passwords"
    },
    {
      "vulnerability": "Insufficient Network Segmentation",
      "severity": "Medium",
      "status": "Vulnerable",
      "details": "Web server can directly reach the database server.",
      "remediation": "Implement proper network segmentation and restrict direct database access from the web server"
    }
  ]
}
```

Figure 86: The rest of the report: the network segmentation finding and its advice.

```
def analyze_governance_failures():
    """Analyze security governance failures"""

    vulnerability_report = load_vulnerability_report()
    attack_log = load_attack_log()
    governance_data = load_governance_data()
    patch_status = load_patch_status()

    failures = []

    # Check for patch management failures
    if vulnerability_report:
        for vuln in vulnerability_report["results"]:
            if (
```

Figure 87: Fix 1: the script now reads "results" instead of "vulnerabilities".

```
(kali@kali)-[~/security_governance_lab]
$ nano analyze_governance_failures.py

(kali@kali)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
Traceback (most recent call last):
  File "/home/kali/security_governance_lab/./analyze_governance_failures.py",
    line 215, in <module>
    main()
  File "/home/kali/security_governance_lab/./analyze_governance_failures.py",
    line 188, in main
    failures = analyze_governance_failures()
  File "/home/kali/security_governance_lab/./analyze_governance_failures.py",
    line 116, in analyze_governance_failures
    and vulnerability_report["summary"]["critical"] > 0
    ~~~~~
KeyError: 'summary'
```

Figure 88: Second run: another error, where the script reads a summary count that my report does not have.

```
# Check vulnerability management
if governance_data and vulnerability_report:
    vuln_policies = [
        p for p in governance_data["policies"]
        if "vulnerabilit" in p["name"].lower()
    ]

    if (
        vuln_policies
        and vulnerability_report["summary"]["critical"] > critical_vulne
    ):
        v for v in vulnerability_report["results"]
        if v.get("severity") == "Critical"
    ]

if vuln_policies and len(critical_vulnerabilities) > 0:0
    ):
    failures.append({
        "category": "Vulnerability Management",
        "description": "Vulnerability Management Policy not effectiv
        "impact": "Critical vulnerabilities not remediated in a time
```

Figure 89: Fix 2: the script now builds its own list of Critical findings from "results" and counts them.

Result

After the fixes the script ran and identified **six governance failures**. It saved the result as a report file named **governance_failure_report_20261008_054906.json**.

```

(kali@kali)-[~/security_governance_lab]
$ nano analyze_governance_failures.py

(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile analyze_governance_failures.py

(kali@kali)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
Governance failure analysis completed. Report saved to governance_failure_report_20261008_054906.json
Summary: 6 governance failures identified
Patch Management: 1
Incident Detection: 1
Policy Implementation: 1
Vulnerability Management: 1
Metrics and Measurement: 1
Authentication and Access Control: 1

(kali@kali)-[~/security_governance_lab]
$

```

Figure 90: The analysis completes: 6 governance failures identified, one in each of six categories.

Failure category	What went wrong, in simple terms	Link to Equifax
Patch Management	A critical flaw in Struts was left unrepaired.	Equifax did not apply the Struts repair for months.
Incident Detection	The simulated attack succeeded and nothing raised an alarm.	The Equifax breach went unnoticed for about 76 days.
Policy Implementation	A patch policy exists, but the flaw is still open.	Equifax had security policies that were not carried out.
Vulnerability Management	Critical vulnerabilities were found but not fixed in good time.	Known weaknesses stayed open too long.
Metrics and Measurement	The numbers (85 per cent patched, 12 days to fix) did not lead to action.	Measurements did not drive improvement or accountability.
Authentication and Access Control	The database used a weak default password.	Weak access controls helped the attackers move around.

Why each failure matters

- **Patch management.** A known flaw with a ready repair is the easiest weakness for an attacker to use. Leaving it open is a choice, even when nobody made that choice on purpose.
- **Incident detection.** Nothing raised an alarm when the simulated attack ran. The longer an attacker stays unseen, the more data they can take.
- **Policy implementation.** A policy that is written down but not carried out gives the organisation false comfort. It also makes it harder to defend itself afterwards, because it shows the risk was understood.
- **Vulnerability management.** Finding weaknesses is not enough. They must be ranked, given an owner and repaired within a set time.

- **Metrics and measurement.** Numbers such as 85 per cent patch compliance are useful only when someone must respond to them. Otherwise they are decoration.
- **Authentication and access control.** A default password means that anyone who reaches the database can log in as its administrator. Strong, unique passwords are one of the cheapest controls available.

5.3 Comparison with the Equifax case

I then wrote a comparison document, **equifax_comparison.md**, which sets the Equifax failures beside the failures in my lab. The screenshots show it open in the editor and then saved (2.8 kilobytes).

```

kati@kati: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 equifax_comparison.md *
# Comparison of Simulated Environment to Equifax Data Breach

## Overview

This report compares the security governance failures identified in our simu>

## Key Governance Failures in Equifax Breach

1. **Patch Management Failure**: Equifax failed to patch a known vulnerabili>
2. **Security Monitoring Failure**: The breach remained undetected for 76 da>
3. **Network Segmentation Failure**: Attackers were able to move laterally w>
4. **Leadership and Accountability Failure**: There was unclear accountabili>
5. **Policy Implementation Failure**: Security policies were not effectively>

## Comparison with Simulated Environment

| Governance Failure | Equifax | Simulated Environment | Similarity |
|---|---|---|---|
| Patch Management | Failed to patch Apache Struts vulnerability for months |>
| Security Monitoring | Failed to detect breach for 76 days | No real-time m>

```

Figure 91: The comparison report: overview and the key governance failures at Equifax.

```

GNU nano 8.6 equifax_comparison.md *
| Patch Management | Failed to patch Apache Struts vulnerability for months |>
| Security Monitoring | Failed to detect breach for 76 days | No real-time m>
| Network Segmentation | Insufficient segmentation allowed lateral movement |>
| Authentication | Weak credentials and access controls | Default credential>
| Policy Implementation | Policies not effectively implemented | Patch and v>
| Metrics and Measurement | Ineffective security metrics | Metrics not driv>

## Lessons Learned

1. **Effective Patch Management is Critical**: Both Equifax and our simulati>
2. **Security Monitoring Must Be Effective**: Without effective security mon>
3. **Network Segmentation Limits Damage**: Proper network segmentation can c>
4. **Policies Require Implementation**: Having security policies is not suff>
5. **Metrics Must Drive Action**: Security metrics should lead to concrete a>

## Conclusion

The simulated environment successfully recreates many of the security govern>

```

Figure 92: The comparison table and the lessons learned.

```
(kali㉿kali)-[~/security_governance_lab]
$ nano equifax_comparison.md

(kali㉿kali)-[~/security_governance_lab]
$ ls -lh equifax_comparison.md
-rw-rw-r-- 1 kali kali 2.8K Oct  8 05:54 equifax_comparison.md

(kali㉿kali)-[~/security_governance_lab]
$
```

Figure 93: The comparison report is saved (2.8 kilobytes).

Failure	Equifax	Simulated lab	Similarity
Patch management	Did not patch Apache Struts for months	Unpatched Struts on the web server	High
Security monitoring	Did not detect the breach for 76 days	No live monitoring to detect the attack	High
Network segmentation	Weak separation allowed attackers to move around	Web server can reach the database directly	High
Authentication	Weak credentials and access controls	Default password on the database	Medium
Policy implementation	Policies not carried out	Policies exist but are not enforced	High
Metrics	Measurements did not drive action	Metrics did not lead to improvement	Medium

5.4 Key governance lessons

The first lesson is that **having a policy is not the same as having control**. In both Equifax and my lab, a patching policy existed, but nothing made sure that the work was done and nobody checked the result. Good governance needs a named owner for each task, a deadline, and a check that reports back to leadership. The second lesson is that **technology alone cannot fix this**. Scanners can find a flaw in seconds, but a person must decide to repair it and a manager must make sure that they do.

The third lesson is about **limiting the damage**. Because the web server could reach the database directly, one weak point led straight to the sensitive data. Splitting the network into zones and using strong passwords would have made the attack much harder. The fourth lesson is that **numbers must lead to action**. My tracker showed 85 per cent patch compliance in red, but nothing in the system forced anyone to respond. Metrics are only useful when someone is responsible for acting on them, and when the dashboard tells the truth.

6. Part 3: Implementing security governance controls

6.1 The control implementation script

I created a script called **implement_governance_controls.py**. It adds five groups of controls and records each one in the governance tracker. I checked its syntax, made it executable and ran it with the option **--all**.

```
(kali@kali)-[~/security_governance_lab]
$ nano equifax_comparison.md

(kali@kali)-[~/security_governance_lab]
$ ls -lh equifax_comparison.md
-rw-rw-r-- 1 kali kali 2.8K Oct  8 05:54 equifax_comparison.md

(kali@kali)-[~/security_governance_lab]
$ cd ~/security_governance_lab

(kali@kali)-[~/security_governance_lab]
$ nano implement_governance_controls.py

(kali@kali)-[~/security_governance_lab]
$ python3 -m py_compile implement_governance_controls.py

(kali@kali)-[~/security_governance_lab]
$ chmod +x implement_governance_controls.py

(kali@kali)-[~/security_governance_lab]
```

Figure 94: Creating the controls script, checking its syntax and making it executable.

```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 implement_governance_controls.py *
#!/usr/bin/env python3

import os
import json
import datetime
import subprocess
import argparse
from governance_tracker import GovernanceTracker

def implement_patch_management():
    """Implement improved patch management controls."""
    print("Implementing patch management controls... ")

    subprocess.run(
        ["/usr/bin/env python3", "--apply", "PATCH-001"],
        check=True
    )

    tracker = GovernanceTracker()

    control_id = tracker.add_control(
        "Automated Patch Management",
        "Automated system for identifying, testing, and applying security patches"
    )
```

Figure 95: The controls script in the nano editor: the patch management control.

```

kali@kali)-[~/security_governance_lab]
$ ./implement_governance_controls.py --all
Implementing patch management controls...
Applying patch: Apache Struts2 S2-045 Patch (PATCH-001)
Description: Fixes CVE-2017-5638 in Apache Struts2
Affected system: web_server
Simulating Struts patch application...
Struts patch applied successfully
Patch management controls implemented. Control ID: CTL-004, Metric ID: MET-004
Implementing network segmentation controls...
Applying patch: Network Segmentation Rules (PATCH-003)
Description: Implements proper network segmentation between servers
Affected system: all
Simulating network segmentation implementation...
Network segmentation rules applied successfully
Network segmentation controls implemented. Control ID: CTL-005
Implementing database security controls...
Applying patch: MySQL Security Configuration (PATCH-002)
Description: Secures MySQL with proper authentication and access controls
Affected system: database_server
Simulating MySQL security configuration...
MySQL security configuration applied successfully
Database security controls implemented. Control ID: CTL-006
Implementing security monitoring controls...
Security monitoring controls implemented. Control ID: CTL-007, Metric ID: MET-005

```

Figure 96: Running the script: patch management (CTL-004) and network segmentation controls are applied.

```

kali@kali: ~/security_governance_lab
Session Actions Edit View Help
Implementing governance oversight controls...
Governance oversight controls implemented. Policy ID: POL-004, Control ID: CTL-008
Verifying security controls...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 4
Controls: 8
Metrics: 5

Security Governance Vulnerability Scanner

[*] Checking Apache Struts S2-045 vulnerability...
[*] Checking MySQL default credentials...
[*] Checking network segmentation...

[+] Report generated: vulnerability_report_20261008_061829.json

SCAN SUMMARY

Apache Struts S2-045
Severity: Critical

```

Figure 97: The oversight policy (POL-004) and control (CTL-008) are added, and the verification begins.

Control area	What it does	Tracker record
Patch management	Applies the Struts patch (simulated) and adds an automated patch process and a patch compliance metric.	CTL-004
Network segmentation	Applies network separation rules (simulated) and records a control based on "least privilege" (give access only where it is needed).	CTL-005

Control area	What it does	Tracker record
Database security	Applies the database security patch (simulated) and records strong login and access controls.	CTL-006
Security monitoring	Creates a monitoring script that looks for suspicious web requests, database access and file changes, plus a schedule to run it every five minutes.	CTL-007
Governance oversight	Creates a document that sets roles and duties for executives and the board, and an oversight policy.	POL-004 and CTL-008

6.2 Verifying the controls

The script ends by checking the results and by running the vulnerability scanner again. The output needs careful reading, because several of the results do not mean what they first appear to mean.

```

SCAN SUMMARY

Apache Struts S2-045
Severity: Critical
Status: Error
Details: Connection error: HTTPConnectionPool(host='172.19.0.2', port=8080):
Max retries exceeded with url: / (Caused by NewConnectionError('<urllib3.conn
nection.HTTPConnection object at 0x7fbb64c6de80>: Failed to establish a new co
nnection: [Errno 113] No route to host'))

MySQL Default Credentials
Severity: High
Status: Not Vulnerable
Details: Default MySQL credentials were not accepted.

Insufficient Network Segmentation
Severity: Medium
Status: Potentially Vulnerable
Details: Database connectivity check returned a non-zero status. Review netwo
rk configuration manually.

Scan complete.

```

Figure 98: The scanner re-run after the controls: Struts shows an error, the database shows Not Vulnerable and network segmentation shows Potentially Vulnerable.

Check	Result	What it really means
Struts patched	False	This does not prove the patch failed. The patch tool overwrites its status file. In this script the database control runs last, so only "MySQL secured" was kept.
MySQL secured	True	Only the most recent patch status survived (see above).
Network segmented	False	Same reason: the status was overwritten.

Check	Result	What it really means
Monitoring script	Implemented	The file was created.
Oversight document	Implemented	The file was created.
Governance tracker	4 policies, 8 controls, 5 metrics	The tracker was updated as intended.
Scan: Struts	Error: no route to host	The scanner could not reach the container. This is a connection problem. It is not proof that the flaw is gone.
Scan: database	Not Vulnerable	This turned out to be misleading. A direct test later showed that the default password still worked (Section 6.3).
Scan: network	Potentially Vulnerable	The database connection test returned a failure code, and the scanner asked for a manual check.

The second attack simulation

The lab asks for the attack to be repeated to see if it now fails. It did fail, with "Connection refused" for localhost on port 8080.

```

kali@kali:~/security_governance_lab$ ./simulate-attack.py
Simulating attack on vulnerable Struts application...
Error simulating attack: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: / (Caused by NewConnectionError(<urllib3.connection.HTTPConnection object at 0x7f6e531ad30>: Failed to establish a new connection: [Errno 111] Connection refused))

```

Figure 99: The second attack simulation ends with "Connection refused" for localhost on port 8080.

How I interpret this result

The attack did not complete, but I cannot say that the controls stopped it. Figure 100, taken during the database password test, shows **docker ps** listing no running containers at that moment. This means the lab containers had stopped at some point and I had to start them again by hand. A stopped or unreachable Struts container would explain both the "Connection refused" message and the "No route to host" message in the final scan.

The screenshots do not show exactly when the containers stopped, so this is the most likely explanation and not a proven one. In addition, the patch tool in this lab only simulates a repair and does not change the real Struts container. A true test of the fix would need a newer, repaired Struts version to be deployed and the same test to be repeated against it.

I therefore treat the failed attack as a **connection problem**, not as proof of success. I believe an honest report is more valuable than an impressive one.

6.3 Database administrator password: testing and fixing

After the controls were applied, the scanner said the default database password was no longer accepted (Figure 98). A scan result is only as good as the scanner, so I tested the claim by hand with a direct login to the database container.

Test 1: the default password still worked

docker ps listed no running containers, so I started the database container again with **docker start database_server**. I then logged in as the administrator using the default password. The

database accepted it and returned the number 1. This proved that the password control had not worked and that the scanner's "Not Vulnerable" result was wrong.

```
(kali@kali)-[~]
$ cd ~/security_governance_lab
(kali@kali)-[~/security_governance_lab]
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
(kali@kali)-[~/security_governance_lab]
$ docker start database_server
database_server
(kali@kali)-[~/security_governance_lab]
$ docker exec -it database_server mysql -uroot -ppassword -e "SELECT 1;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+
| 1 |
+----+
| 1 |
+----+
(kali@kali)-[~/security_governance_lab]
$
```

Figure 100: *docker ps* lists no running containers, so *database_server* is started by hand. A direct login with the default password then succeeds and returns 1.

Why did the scanner get it wrong? The most likely reason is that the scanner treats any failed login as "Not Vulnerable". If the database container was stopped when the scan ran, the login would fail because nothing was there to answer, and the scanner would wrongly report that the credentials were refused. The screenshots do not prove the timing, but the lesson is clear: a good scanner must tell "could not connect" apart from "login refused".

The fix: changing the administrator password

I changed the administrator password with an **ALTER USER** command followed by **FLUSH PRIVILEGES**, which makes the database use the new password straight away. I then tried the default password again. This time the database refused it with "ERROR 1045 (28000): Access denied".

```
(kali@kali)-[~/security_governance_lab]
$ docker exec -it database_server mysql -uroot -ppassword -e "ALTER USER 'root'@'localhost' IDENTIFIED BY 'password'; FLUSH PRIVILEGES;"
mysql: [Warning] Using a password on the command line interface can be insecure.
(kali@kali)-[~/security_governance_lab]
$ docker exec -it database_server mysql -uroot -ppassword -e "SELECT 1;"
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password : YES)
(kali@kali)-[~/security_governance_lab]
$
```

Figure 101: *The default administrator password is changed with an ALTER USER command. Trying the default password straight afterwards is rejected with "Access denied".*

A note about passwords on the command line

The database program itself warns, in every one of these screenshots, that using a password on the command line can be insecure. The new password in Figure 101 was blurred to preserve the password.

Test 2: the new password works and the default does not

To prove the change, I logged in again using the hidden prompt, which asks for the password without showing it on screen. The first two attempts were rejected and the third was accepted and returned 1. I then tested the default password once more and it was rejected again.

```
(kali@kali)-[~/security_governance_lab]
$ docker exec -it database_server mysql -uroot -p -e "SELECT 1;"
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password : YES)

(kali@kali)-[~/security_governance_lab]
$ docker exec -it database_server mysql -uroot -p -e "SELECT 1;"
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password : YES)

(kali@kali)-[~/security_governance_lab]
$ docker exec -it database_server mysql -uroot -p -e "SELECT 1;"
Enter password:
+----+
| 1 |
+----+
+----+
| 1 |
+----+
```

Figure 102: Logging in with the password typed at the prompt. The first two attempts are rejected with "Access denied". The third is accepted and returns 1.

```
(kali@kali)-[~/security_governance_lab]
$ docker exec -it database_server mysql -uroot -ppassword -e "SELECT 1;"
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password : YES)
```

Figure 103: The default password is tested again and is rejected with "Access denied".

Test	Result	What it shows
Default password, before the change	Accepted (returned 1)	The control was not in place, and the scanner's earlier result was wrong.
Default password, straight after the change	Rejected (Access denied)	The default password stopped working.
New password, typed at the prompt	Third attempt accepted (returned 1)	The new password works.
Default password, tested again	Rejected (Access denied)	The change is lasting.

What these tests prove, and what they do not

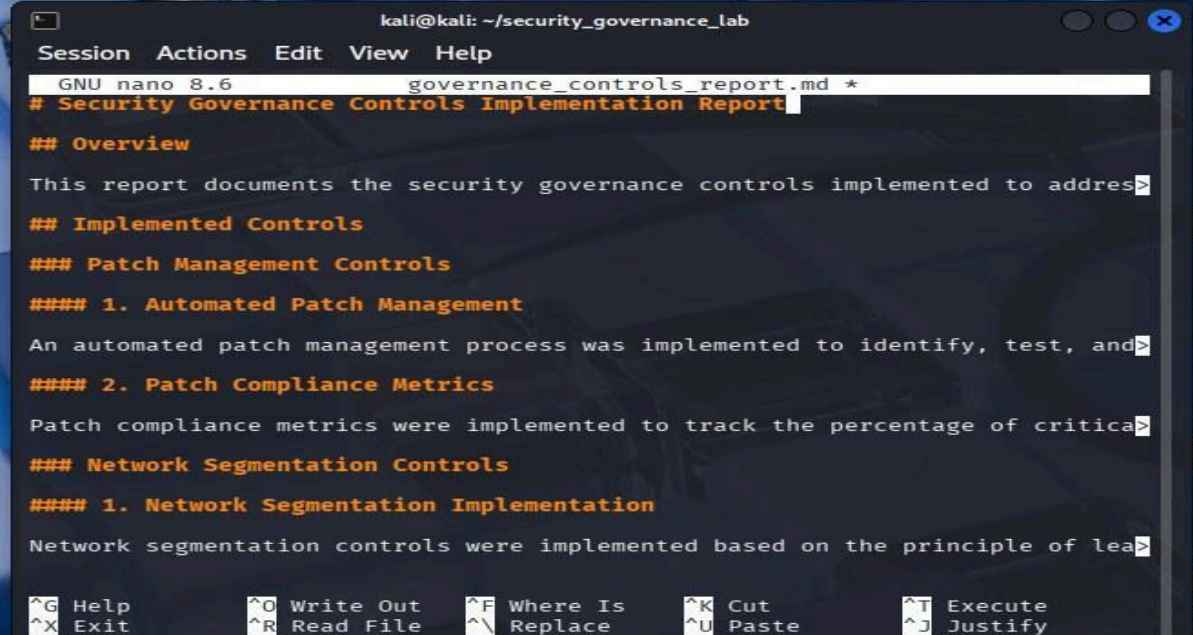
They prove that, for the administrator account called root on the local machine inside the database container, the default password no longer works and the new one does. This is direct evidence, and it is the strongest evidence in the whole lab.

They do not show three other things. First, MySQL can hold several administrator accounts for different places, and I only changed one. Second, the application user (**app_user**) still has a weak password set in the compose file. Third, the compose file itself still contains the old administrator password, so if the database container were rebuilt from scratch the weak

password could return. I also did not re-run the scanner after the change, so there is no new scan report to show.

6.4 The controls report

I wrote **governance_controls_report.md** (3.9 kilobytes, saved at 06:25) to document the controls. In it I took care not to claim that the Struts fix was proved. The conclusion says that several controls work as intended and that the Struts result still needs confirming.



```
kali@kali: ~/security_governance_lab
Session Actions Edit View Help
GNU nano 8.6 governance_controls_report.md *
# Security Governance Controls Implementation Report

## Overview
This report documents the security governance controls implemented to address>
## Implemented Controls
### Patch Management Controls
#### 1. Automated Patch Management
An automated patch management process was implemented to identify, test, and>
#### 2. Patch Compliance Metrics
Patch compliance metrics were implemented to track the percentage of critica>
### Network Segmentation Controls
#### 1. Network Segmentation Implementation
Network segmentation controls were implemented based on the principle of lea>
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify
```

Figure 104: Writing *governance_controls_report.md* in the nano editor.



```
(kali@kali)-[~/security_governance_lab]
$ cd ~/security_governance_lab

(kali@kali)-[~/security_governance_lab]
$ nano governance_controls_report.md

(kali@kali)-[~/security_governance_lab]
$ ls -lh governance_controls_report.md
-rw-rw-r-- 1 kali kali 3.9K Oct  8 06:25 governance_controls_report.md

(kali@kali)-[~/security_governance_lab]
$
```

Figure 105: The control report is saved (3.9 kilobytes).

```
Session Actions Edit View Help
kali@kali: ~/security_governance_lab
$ cat governance_controls_report.md
# Security Governance Controls Implementation Report

## Overview

This report documents the security governance controls implemented to address the governance failures identified in the simulated environment.

## Implemented Controls

### Patch Management Controls

#### 1. Automated Patch Management

An automated patch management process was implemented to identify, test, and apply security patches. The critical Apache Struts vulnerability was specifically addressed, with clear ownership and accountability for patching activities.

#### 2. Patch Compliance Metrics

Patch compliance metrics were implemented to track the percentage of critical patches that have been applied. The target is 100% of critical patches within 7 days, with regular reporting to management.

### Network Segmentation Controls

#### 1. Network Segmentation Implementation

Network segmentation controls were implemented based on the principle of least privilege. Direct access between the web server and database server is restricted to reduce the possibility of lateral movement following a compromise.

### Database Security Controls

#### 1. Database Authentication and Access Controls

Database security controls were implemented to require strong authentication and restrict database access to authorized users and applications. Default database credentials are removed or secured.

### Security Monitoring Controls

#### 1. Real-time Security Monitoring
```

Figure 106: The controls report displayed: overview and patch management controls.

```
Session Actions Edit View Help
#### 1. Real-time Security Monitoring

Real-time security monitoring was implemented to identify suspicious activity and provide automated alerting. Security monitoring and regular security reports support ongoing detection and response.

#### 2. Security Monitoring Metrics

Security monitoring coverage was established as a measurable security metric. The target is 100% coverage of critical systems, with regular reporting and review.

### Governance Oversight Controls

#### 1. Security Governance Oversight Policy

A security governance oversight policy was implemented to define security roles and responsibilities, executive and board oversight, and security reporting and escalation requirements.

#### 2. Executive Security Oversight

Executive security oversight was implemented through regular security briefings, accountability mechanisms, and management review of security risks and controls.

## Verification Results

The implemented controls were verified through the following activities:

### 1. Vulnerability Scanning

- Vulnerability scanning was performed after control implementation.
- Database default credentials were no longer accepted.
- Network segmentation was checked through database connectivity testing.
- The Apache Struts vulnerability scan encountered a connectivity error, so remediation of the Struts vulnerability could not be conclusively verified by the final scan.

### 2. Control Testing

- Patch management controls were implemented and tested.
- Network segmentation controls were implemented.
- Database security controls were implemented and tested.
- Security monitoring controls were implemented.
- Governance oversight documentation was implemented.
```

Figure 107: The controls report continues: security monitoring and governance oversight controls.

```
Session Actions Edit View Help
- Security monitoring controls were implemented.
- Governance oversight documentation was implemented.

### 3. Governance Verification

The governance tracker was updated with the newly implemented controls.

The final governance tracker contained:
- 4 policies
- 8 controls
- 5 metrics

## Conclusion

The implemented controls address the major security governance failures identified during the simulation. The controls establish stronger patch management, network segmentation, database security, security monitoring, and governance oversight.

The verification results demonstrate that several controls are functioning as intended. However, the Apache Struts vulnerability requires additional verification because the final vulnerability scan encountered a connectivity error. This highlights the importance of continuous monitoring, verification, and accountability in security governance.

(kali@kali)-[~/security_governance_lab]
```

Figure 108: The end of the controls report: verification results, tracker totals and the conclusion.

A listing of the files from Parts 2 and 3 confirms that all four required items exist: the attack log, the comparison report, the controls report and the failure report.

```
(kali@kali)-[~/security_governance_lab]
$ ls -lh attack_log.json governance_failure_report*.json equifax_comparison.md governance_controls_report.md
-rw-rw-r-- 1 kali kali 209 Oct  8 04:49 attack_log.json
-rw-rw-r-- 1 kali kali 2.8K Oct  8 05:54 equifax_comparison.md
-rw-rw-r-- 1 kali kali 3.9K Oct  8 06:25 governance_controls_report.md
-rw-rw-r-- 1 kali kali 2.1K Oct  8 05:49 governance_failure_report_20261008_054906.json
```

Figure 109: The deliverables from Parts 2 and 3: attack_log.json, equifax_comparison.md, governance_controls_report.md and the governance failure report.

```
Session Actions Edit View Help
~$ ./simulation_governance_controls.py --verify
Verifying security controls...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 4
Controls: 8
Metrics: 5

Security Governance Vulnerability Scanner

[*] Checking Apache Struts S2-045 vulnerability...
[*] Checking MySQL default credentials...
[*] Checking network segmentation...

[*] Report generated: vulnerability_report_20261008_063609.json

SCAN SUMMARY

Apache Struts S2-045
Severity: Critical
Status: Error
Details: Connection error: HTTPConnectionPool(host='172.19.0.2', port=8080): Max retries exceeded with url: / (Caused by NewConnectionError('<urllib3.connection.HTTPConnection object at 0x7f66c3755e80>: Failed to establish a new connection: [Errno 113] No route to host'))

MySQL Default Credentials
Severity: High
Status: Not Vulnerable
Details: Default MySQL credentials were not accepted.

Insufficient Network Segmentation
Severity: Medium
Status: Potentially Vulnerable
Details: Database connectivity check returned a non-zero status. Review network configuration manually.

Scan complete.

(kali@kali)-[~/security_governance_lab]
```

Figure 110: A second verification run shows the same results, and a new scan report (vulnerability_report_20261008_063609.json).

6.5 How the controls address the Equifax failures

Each control answers one of the failures found in Part 2. The **patch control** and its metric aim to make sure that critical repairs are applied within seven days, which is the failure that started the Equifax breach. **Network segmentation** limits how far an attacker can go once inside, which answers the weak separation between web server and database. **Database security** removes default passwords, which closes an easy route to the data. **Security monitoring** is meant to give an early warning, so that an attack is noticed in minutes and not after 76 days.

The **governance oversight** controls answer the deepest failure: unclear responsibility. They name the executive who is responsible for each area, require regular briefings to the board and tie performance to security duties. Together, the controls cover technology (patching, network, database, monitoring) and people and process (oversight, owners and reporting). That mix is what the Equifax case shows is needed. However, as Section 6.7 explains, in this lab most controls were recorded and simulated, so they show the design of a good response and not yet its proven effect.

6.6 From failure to control to metric

A strong governance system links every failure to a control, every control to a number that proves it works, and every number to a named owner. The table below shows those links for my lab. The owners are suggestions of the kind of role that would be responsible in a real organisation.

Failure found	Control that answers it	Metric that proves it	Suggested owner (role)
Patch management	CTL-004 Automated patch management	Patch compliance	Chief Information Officer
Vulnerability management	CTL-004 and CTL-005	Vulnerability remediation time	Chief Information Security Officer
Incident detection	CTL-007 Security monitoring	Monitoring coverage and security incidents	Security operations manager
Policy implementation	CTL-008 Oversight and POL-004	Policy compliance	Chief Information Security Officer
Metrics and measurement	CTL-008 Oversight	All five metrics and the board report	Board risk committee
Authentication and access	CTL-006 Database security	None yet	Database owner
Network separation	CTL-005 Network segmentation	None yet	Head of network services

The table exposes a gap. The two controls that deal with passwords and network separation have **no metric** in my dashboard, so nobody would see if they slipped. I recommend adding

two metrics: the share of accounts still using default or weak passwords, and the number of paths that allow the web server to reach the database directly.

6.7 How strong is the evidence for each control?

Not every control is proved to the same level. This table rates the evidence in my screenshots in plain terms. "Strong" means I tested the real system directly. "Partial" means something was created but its effect was not tested. "Weak" means the result was simulated or the check could not connect.

Control	What the screenshots show	Strength of evidence
Database password (CTL-006)	Direct tests: default password rejected, new password accepted (Figures 100 to 103).	Strong
Struts patch (CTL-004)	A simulated status message only. The final scan could not connect to the container, and the second attack failed with a connection error.	Weak
Network segmentation (CTL-005)	A simulated status message. The scanner still says "Potentially Vulnerable".	Weak
Security monitoring (CTL-007)	The monitoring script and its schedule were created. No screenshot shows it running or raising an alert.	Partial
Governance oversight (CTL-008 and POL-004)	The oversight document and policy exist, and the tracker shows 4 policies and 8 controls. Their real effect depends on people following them.	Partial

7. Part 4: Security governance metrics and reporting

7.1 The metrics script

The script **governance_metrics.py** creates six months of sample data for five metrics. For the first five months it uses random numbers in a realistic range. For the current month it sets the value to the ideal figure. It then draws a chart for each metric, builds an executive dashboard (a web page) and writes a board report.

```

Session Actions Edit View Help
GNU nano 8.6 governance_metrics.py *
#!/usr/bin/env python3

import os
import datetime
import random
import matplotlib.pyplot as plt

def generate_historical_data():
    """Generate historical data for security governance metrics."""
    months = []
    current_month = datetime.datetime.now()

    # Generate 6 months of historical data
    for i in range(6):
        month = current_month - datetime.timedelta(days=30 * i)
        months.insert(0, month.strftime("%b %Y"))

    # Patch compliance
    patch_compliance = [random.uniform(70, 85) for _ in range(5)]
    patch_compliance.append(100.0)

    # Vulnerability remediation time
    vuln_remediation = [random.uniform(10, 15) for _ in range(5)]
    vuln_remediation.append(1.0)

    # Security incidents
    security_incidents = [random.randint(1, 3) for _ in range(5)]
    security_incidents.append(0)

    # Security monitoring coverage
    monitoring_coverage = [random.uniform(60, 80) for _ in range(5)]
    monitoring_coverage.append(100.0)

    # Policy compliance
    policy_compliance = [random.uniform(75, 90) for _ in range(5)]
    policy_compliance.append(100.0)

    return {
        "months": months,
        "patch_compliance": patch_compliance,

```

Figure 111: The first part of `governance_metrics.py`: the function that creates six months of sample data.

```

Session Actions Edit View Help
GNU nano 8.6 governance_metrics.py *
$. Continue tracking security metrics over time.

## Conclusion
The security governance metrics demonstrate significant improvement in the simulated environment. Continued measurement, reporting, and executive oversight will support ongoing security governance and continuous

...

with open("board_report.md", "w") as f:
    f.write(report_md)

return "board_report.md"

def main():
    """Generate all security governance metrics and reports."""
    data = generate_historical_data()
    visualization_files = generate_metrics_visualizations(data)
    dashboard_file = generate_executive_dashboard(
        data,
        visualization_files
    )

    board_report_file = generate_board_report(
        data,
        visualization_files
    )

    print("Security governance metrics and reporting generated:")
    print(f"- Executive Dashboard: {dashboard_file}")
    print(f"- Board Report: {board_report_file}")
    print(
        f"- Visualizations: "
        f"{', '.join(visualization_files)}"
    )

if __name__ == "__main__":
    main()

```

Figure 112: The end of the metrics script: it writes `board_report.md` and runs all the steps.

```

(kali@kali)~/security_governance_lab
$ cd ~/security_governance_lab
(kali@kali)~/security_governance_lab
$ nano governance_metrics.py
(kali@kali)~/security_governance_lab
$ python3 -m py_compile governance_metrics.py
(kali@kali)~/security_governance_lab
$ chmod +x governance_metrics.py
(kali@kali)~/security_governance_lab
$ ./governance_metrics.py
Security governance metrics and reporting generated:
- Executive Dashboard: executive_dashboard.html
- Board Report: board_report.md
- Visualizations: metrics_visualizations/patch_compliance.png, metrics_visualizations/vuln_remediation.png, metrics_visualizations/security_incidents.png, metrics_visualizations/monitoring_coverage.png, metrics_visualizations/policy_compliance.png
(kali@kali)~/security_governance_lab
$

```

Figure 113: Checking the script, making it executable and running it. It creates the dashboard, the board report and five charts.

```
(kali@kali)-[~/security_governance_lab]
$ ls -lh executive_dashboard.html board_report.md metrics_visualizations/
-rw-rw-r-- 1 kali kali 1.9K Oct  8 06:45 board_report.md
-rw-rw-r-- 1 kali kali 4.0K Oct  8 06:45 executive_dashboard.html

metrics_visualizations/:
total 232K
-rw-rw-r-- 1 kali kali 44K Oct  8 06:45 monitoring_coverage.png
-rw-rw-r-- 1 kali kali 41K Oct  8 06:45 patch_compliance.png
-rw-rw-r-- 1 kali kali 42K Oct  8 06:45 policy_compliance.png
-rw-rw-r-- 1 kali kali 45K Oct  8 06:45 security_incidents.png
-rw-rw-r-- 1 kali kali 51K Oct  8 06:45 vuln_remediation.png

(kali@kali)-[~/security_governance_lab]
$
```

Figure 114: The files created: board_report.md, executive_dashboard.html and the metrics_visualizations folder with five charts.

Metric	Chart file	Target	Current month (sample)	Better when
Patch compliance	patch_compliance.png	95 per cent	100 per cent	Higher
Vulnerability remediation time	vuln_remediation.png	7 days	1 day	Lower
Security incidents	security_incidents.png	0	0	Lower
Security monitoring coverage	monitoring_coverage.png	95 per cent	100 per cent	Higher
Policy compliance	policy_compliance.png	95 per cent	100 per cent	Higher

7.2 The executive dashboard

The dashboard is a single web page for senior leaders. It opens in any web browser and was generated on 8 October 2026 at 06:45.

```
(kali@kali)-[~/security_governance_lab]
$ xdg-open executive_dashboard.html
```

Figure 115: Opening the dashboard in the web browser with the xdg-open command.

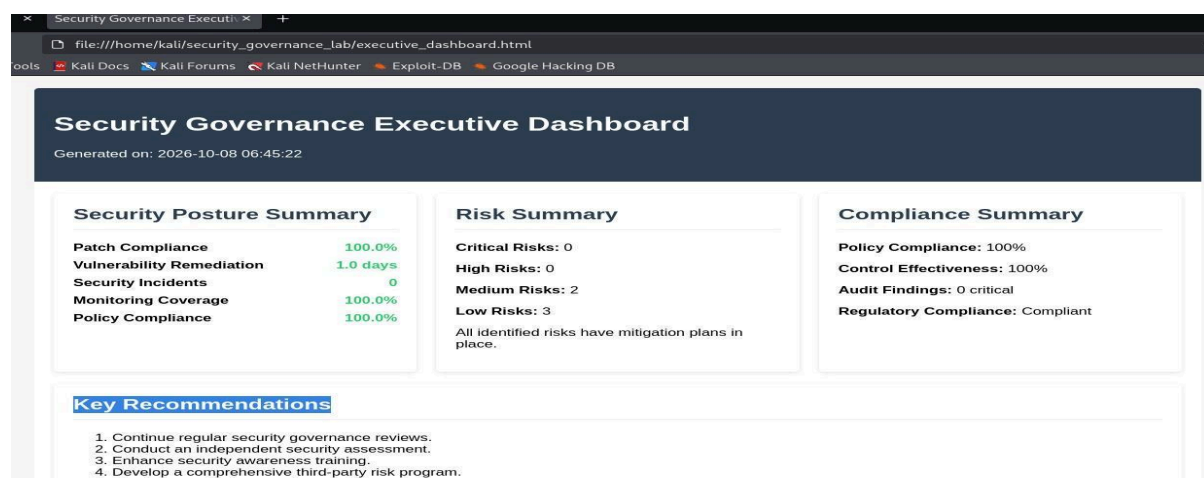


Figure 116: The Security Governance Executive Dashboard in the browser: posture, risk and compliance panels and key recommendations.

Panel	What it says
Security Posture Summary	Patch compliance 100.0 per cent, vulnerability remediation 1.0 days, security incidents 0, monitoring coverage 100.0 per cent and policy compliance 100.0 per cent.
Risk Summary	Critical risks 0, high risks 0, medium risks 2 and low risks 3. A line says all identified risks have mitigation plans in place.
Compliance Summary	Policy compliance 100 per cent, control effectiveness 100 per cent, 0 critical audit findings and regulatory compliance shown as "Compliant".
Key Recommendations	Five items: continue regular governance reviews, carry out an independent security assessment, improve security awareness training, build a full third-party risk programme and keep monitoring the metrics.

The dashboard and the tracker disagree

The governance tracker in Part 1 recorded three risks: one critical, one high and one medium (Figure 67). The Part 4 dashboard shows five risks: none critical, none high, two medium and three low. It also reports control effectiveness of 100 per cent and regulatory compliance as "Compliant", even though the Struts control could not be proved.

The reason is that the Part 4 script makes its own sample numbers and does not read the tracker or the scan reports. This is a useful warning. A dashboard that is not fed from trusted evidence can look excellent while the real position is poor.

7.3 The five trend charts

Each chart shows one metric over six months, with a dashed line for the target. A file manager window confirms that all five chart files were created, and each chart is shown in full below.

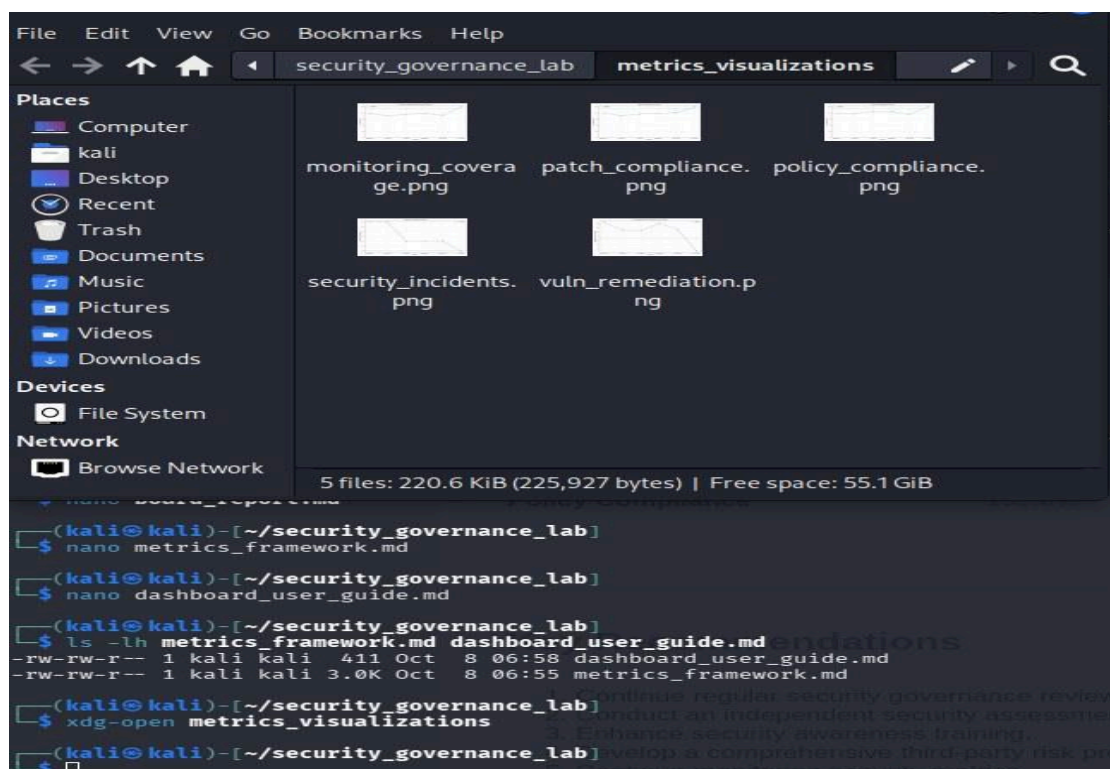


Figure 117: The five chart files in the `metrics_visualizations` folder, shown in the file manager.

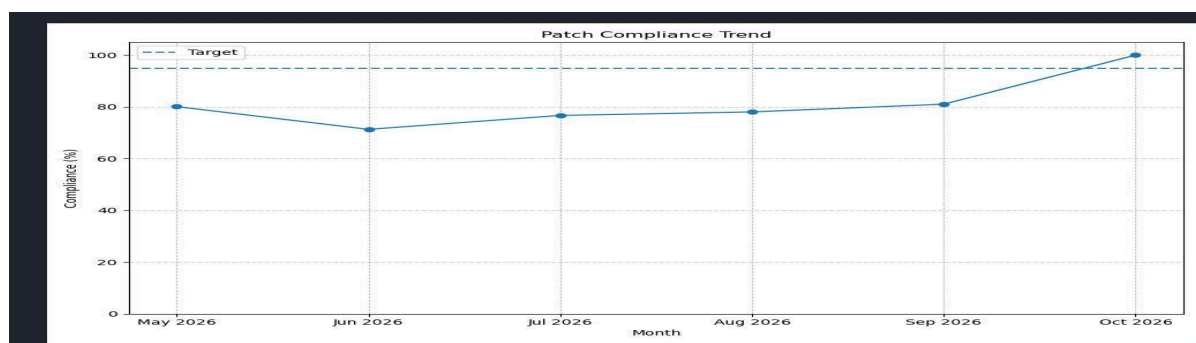


Figure 118: Patch Compliance Trend: about 71 to 81 per cent for five months, then 100 per cent in October 2026, above the 95 per cent target.

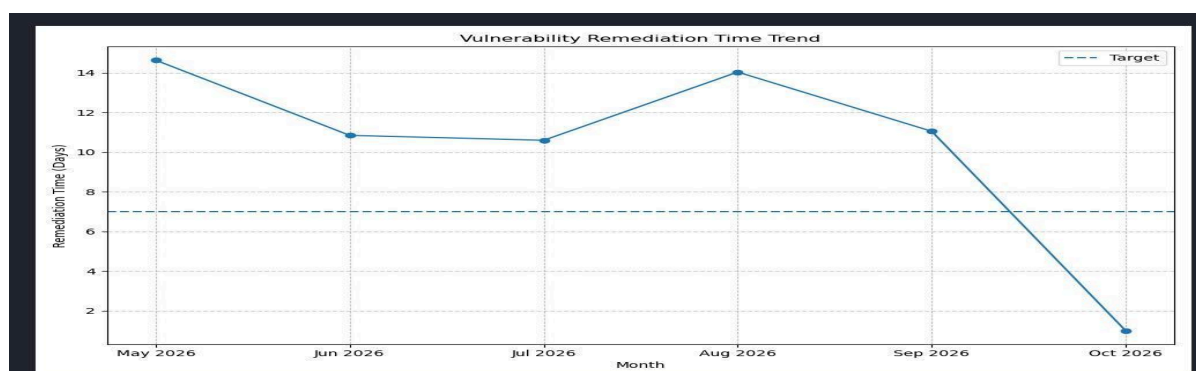


Figure 119: Vulnerability Remediation Time Trend: about 11 to 15 days for five months, above the 7-day target, then 1 day in October 2026.

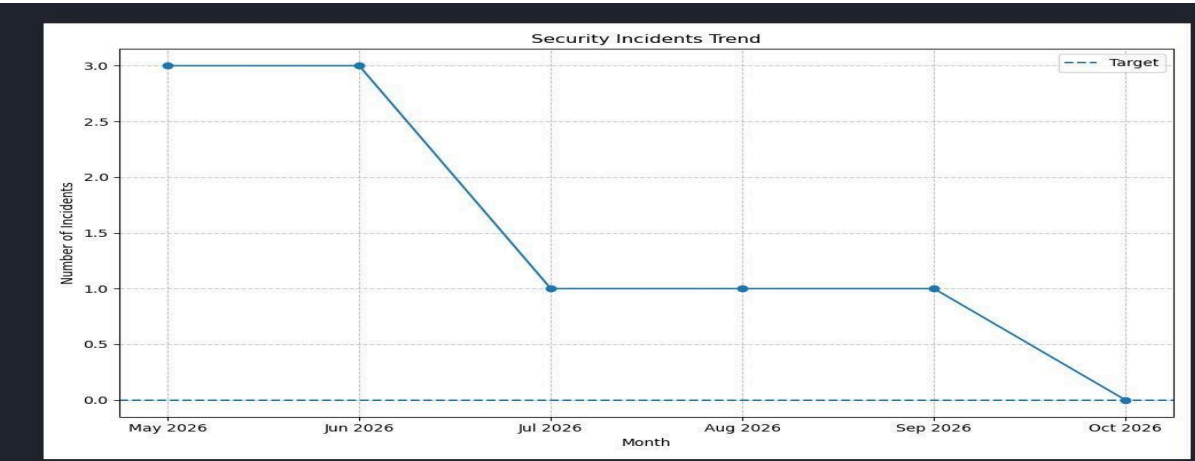


Figure 120: Security Incidents Trend: 3 incidents in May and June, 1 from July to September, then 0 in October 2026, which meets the target.

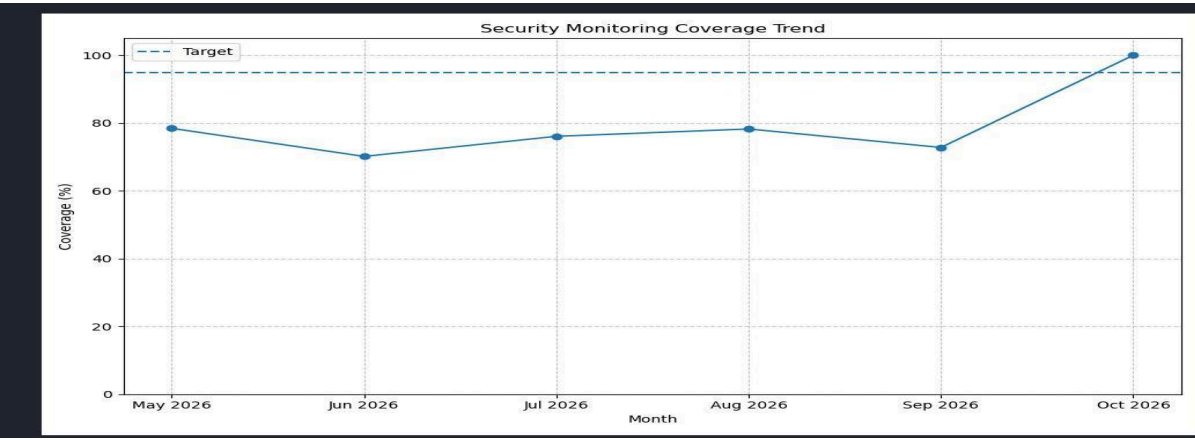
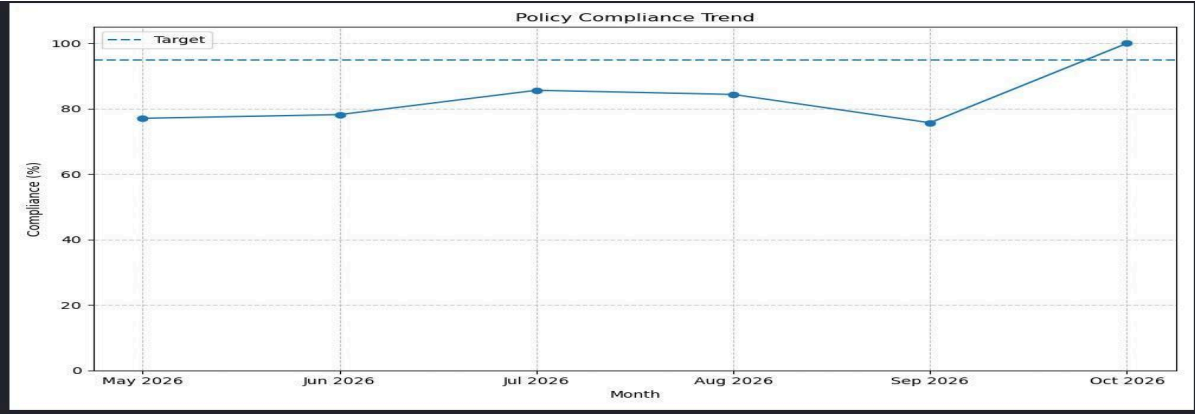


Figure 121: Security Monitoring Coverage Trend: about 70 to 79 per cent for five months, then 100 per cent in October 2026.

Figure 122: Policy Compliance Trend: about 76 to 86 per



cent for five months, then 100 per cent in October 2026.

Metric	May to September (approximately)	October	Target	What to notice
Patch compliance	71 to 81 per cent	100 per cent	95 per cent	Below target for five months, then a sudden jump over the line.

Metric	May to September (approximately)	October	Target	What to notice
Remediation time	11 to 15 days	1 day	7 days	Slower than target for five months. This is the only chart where lower is better, so the line must fall below the target to succeed.
Security incidents	3, 3, 1, 1, 1	0	0	A steady fall. The target of zero is reached only in October.
Monitoring coverage	70 to 79 per cent	100 per cent	95 per cent	No month reaches the target until October.
Policy compliance	76 to 86 per cent	100 per cent	95 per cent	Closest to target in July, then falls in September before the jump.

The values above are read from the charts, so they are approximate. All five charts tell the same story: months of performance below target, followed by a sudden jump to perfection in the last month. Real security figures almost never behave like this. A board member would be right to ask what changed overnight. In this lab the answer is simple: the script sets the last month to the ideal value. The charts show how trend reporting works, and not what happened in the lab.

The charts also show why direction matters. For three metrics higher is better, and for two (remediation time and incidents) lower is better. A reader must always check which way is good before judging a line, and a well-made dashboard should make that obvious.

7.4 The board report, metrics framework and user guide

The **board report** is a short written summary for the directors. It gives an executive summary, the key metrics with their status, the risks, the improvements made and recommendations. The **metrics framework** explains each metric: what it means, its target, how often it is reported, who owns it and who reads it. The **dashboard user guide** explains how to open the dashboard and how to read the colours.

```

(kali@kali) (~/.security_governance_lab)
$ cat board_report.md

# Security Governance Board Report

## Executive Summary
The organization's security governance posture has improved significantly following implementation of the security governance controls.

## Key Metrics
| Metric | Current | Target | Status |
| :--- | :--- | :--- | :--- |
| Patch Compliance | 100.0% | 95% | Green |
| Vulnerability Remediation Time | 1.0 days | 7 days | Green |
| Security Incidents | 0 | 0 | Green |
| Security Monitoring Coverage | 100.0% | 95% | Green |
| Policy Compliance | 100.0% | 95% | Green |

## Risk Summary
The organization's risk posture has improved significantly.
- 0 Critical Risks
- 0 High Risks
- 2 Medium Risks
- 3 Low Risks
All identified risks have mitigation plans in place.

## Compliance Status
The organization is fully compliant with applicable internal security policies based on the simulated metrics.

## Security Program Improvements
1. **Enhanced Patch Management:** Automated patch management implemented.
2. **Improved Network Segmentation:** Network segmentation implemented.
3. **Strengthened Database Security:** Strong authentication and access controls implemented.
4. **Enhanced Security Monitoring:** Real-time monitoring and alerting implemented.
5. **Improved Governance Oversight:** Executive and board oversight established.

```

Figure 123: The board report: executive summary, key metrics (all Green) and the risk summary.

```

## Security Program Improvements
1. **Enhanced Patch Management:** Automated patch management implemented.
2. **Improved Network Segmentation:** Network segmentation implemented.
3. **Strengthened Database Security:** Strong authentication and access controls implemented.
4. **Enhanced Security Monitoring:** Real-time monitoring and alerting implemented.
5. **Improved Governance Oversight:** Executive and board oversight established.

## Recommendations
1. Continue the current security governance program.
2. Conduct an independent security assessment.
3. Enhance security awareness activities.
4. Develop a comprehensive third-party risk management program.
5. Continue tracking security metrics over time.

## Conclusion
The security governance metrics demonstrate significant improvement in the simulated environment. Continued measurement, reporting, and executive oversight will support ongoing security governance and continuous improvement.

```

Figure 124: The rest of the board report: security improvements, recommendations and the conclusion.

```

Session Actions Edit View Help
GNU nano 3.6 metrics_framework.md

## 1. Purpose
This framework defines the security governance metrics used to measure security performance, risk reduction, compliance, and continuous improvement.

## 2. Key Metrics


| Metric                         | Description                                                         | Target | Frequency | Owner               | Audience        |
|--------------------------------|---------------------------------------------------------------------|--------|-----------|---------------------|-----------------|
| Patch Compliance               | Percentage of systems with required security patches applied        | 95%    | Monthly   | Security Team       | Management      |
| Vulnerability Remediation Time | Average time required to remediate identified vulnerabilities       | 7 days | Monthly   | Security Team       | Management      |
| Security Incidents             | Number of security incidents during the reporting period            | 0      | Monthly   | Security Team       | Executive/Board |
| Security Monitoring Coverage   | Percentage of systems covered by security monitoring                | 95%    | Monthly   | Security Operations | Management      |
| Policy Compliance              | Percentage of applicable security policies successfully implemented | 95%    | Monthly   | Governance Team     | Executive/Board |



## 3. Reporting Structure
### Executive Dashboard
The executive dashboard provides:
- Security posture summary
- Key metrics and status indicators
- Trend visualizations
- Risk summary
- Compliance summary
- Key recommendations

### Board Report
The board report provides:
- Executive summary
- Key metrics and status
- Risk summary
- Security incidents
- Compliance status
- Security program improvements
- Recommendations
- Conclusion

```

Figure 125: The metrics framework: purpose and the table of key metrics with targets, frequency, owner and audience.

```

Session Actions Edit View Help
GNU nano 3.6 dashboard_user_guide.md

## 1. Purpose
The Security Governance Dashboard provides an executive-level view of the organization's security posture, risks, compliance status, and security governance performance.

## 2. Opening the Dashboard
To open the dashboard:
1. Navigate to the security governance lab directory.
2. Run the security metrics script:
   $ bash ./governance_metrics.py

```

Figure 126: The dashboard user guide: its purpose and how to open the dashboard.

```

(kali@kali)-[~/security_governance_lab]
$ nano board_report.md

(kali@kali)-[~/security_governance_lab]
$ nano metrics_framework.md

(kali@kali)-[~/security_governance_lab]
$ nano dashboard_user_guide.md

(kali@kali)-[~/security_governance_lab]
$ ls -lh metrics_framework.md dashboard_user_guide.md
-rw-rw-r-- 1 kali kali 411 Oct  8 06:58 dashboard_user_guide.md
-rw-rw-r-- 1 kali kali 3.0K Oct  8 06:55 metrics_framework.md

```

Figure 127: Creating the board report, metrics framework and user guide in the nano editor, and checking their sizes.

```
(kali@kali)-[~/security_governance_lab]
$ ls -lh governance_metrics.py executive_dashboard.html board_report.md metrics_framework.md dashboard_user_guide.md
-rw-rw-r-- 1 kali kali 1.9K Oct  8 06:45 board_report.md
-rw-rw-r-- 1 kali kali 411 Oct  8 06:58 dashboard_user_guide.md
-rw-rw-r-- 1 kali kali 4.0K Oct  8 06:45 executive_dashboard.html
-rwxrwxr-x 1 kali kali 12K Oct  8 06:43 governance_metrics.py
-rw-rw-r-- 1 kali kali 3.0K Oct  8 06:55 metrics_framework.md
```

Figure 128: All the Part 4 files: the metrics script, dashboard, board report, metrics framework and user guide.

7.5 How metrics and reporting support good governance

Governance depends on leaders being able to see what is happening and to hold people to account. Metrics turn a complicated technical situation into a few clear numbers. The dashboard lets an executive see in seconds whether patching is on target and whether risks are falling. The board report gives directors the information they need to ask good questions, such as why remediation took 12 days when the target was 7. The metrics framework gives every number an owner, a target and a schedule, so that no figure is left without someone responsible. Trend charts matter because they show direction as well as position: a figure that is improving month by month tells a different story from one that is stuck.

These tools work only if they are **honest and based on evidence**. This lab showed two risks. First, the tracker marked some "lower is better" numbers Green when they missed their targets (Section 4.9). Second, the Part 4 dashboard is built from sample numbers, so it shows 100 per cent patch compliance even though the Struts fix could not be confirmed. In a real organisation, every figure on a dashboard should come from a trusted source such as a scan result, and the colours should follow rules that are checked.

8. Problems I encountered and how I solved them

Working through the lab involved a number of faults. While these challenges were so frustrating, solving them was part of the learning, so I list them here.

Problem	Cause	Solution
Docker not found	Docker was not installed on the Kali machine.	Installed the Docker command tool and the Docker engine, started the service and tested with hello-world.
Compose command missing	The compose tool and plug-in package were not available.	Downloaded Docker Compose version 2 as a standalone file and made it executable. Later I also installed the docker-compose package.
Out-of-date "version" warning	Newer compose tools ignore the "version" line in docker-compose.yml.	The warning is harmless. I recommend removing the line.
Docker needed administrator rights	My user was not in the docker group.	Added the user to the docker group so docker ps runs without sudo.

Problem	Cause	Solution
Struts image name differed from the lab sheet	The vulhub s2-045 folder uses vulhub/struts2:2.3.30, and the container is called s2-045-struts2-1.	Used the vulhub environment and updated names and addresses in the scripts. Documented the difference in Section 3.
Scanner could not find the Struts container	Container names only work inside the Docker network. The scanner runs on the Kali machine.	Changed the scanner to use the address 172.19.0.2.
Scanner decoding error on the network check	MySQL returned binary data, which Python tried to read as text.	Removed the text option so the answer is read as raw data.
Patch status overwritten	The patch tool rewrites the status file for each patch.	Understood and documented. Recommended fix: read the file first and update only one entry.
Containers not running	At one point docker ps listed no running containers.	Started database_server again by hand. Recommended: set containers to restart automatically and check they are running before every test.
Attack script could not connect	The internal address was not reliably reachable from Kali.	Changed the target to localhost:8080, which was reachable.
Analysis script stopped (first error)	The script expected "vulnerabilities" but my report uses "results".	Changed the script to read "results".
Analysis script stopped (second error)	The script read a summary count that my report does not contain.	Built the list of critical findings directly from "results".
Scan said the database password was safe, but it was not	The scanner most likely counts a failed login as "Not Vulnerable", and the database container had probably stopped.	Tested by hand, changed the administrator password and tested again (Section 6.3). Recommended: make the scanner tell "could not connect" apart from "login refused".
Final Struts scan could not connect	No route to the container address, probably because the containers had stopped.	Reported as a connection problem. I did not claim that the flaw was fixed.

9. What the evidence shows and does not show

I want this report to be accurate, so this section is plain about the limits of my results.

- **The Struts flaw was not confirmed by an attack.** The scanner saw a normal reply and asked for manual checking. The software version (2.3.30) is known to be affected, but the scan result itself is "Potentially Vulnerable".
- **The attack was a simulation.** The script prints messages and writes a log. No data was really taken.

- **The patching in this lab is simulated.** The patch tool only writes a status file. It does not update the Struts container.
- **The Struts and network results after the controls are unproved.** The Struts check could not connect, the "patched" and "segmented" flags were wrong because of the overwrite fault, and the second attack failed because it could not connect. None of these proves that the system is now safe.
- **The database password is proved, but only in part.** Direct tests show the default administrator password for the local root account was replaced. I did not check other administrator accounts, the application user password or the compose file, and I did not re-run the scanner afterwards.
- **The scanner can give false comfort.** It reported the default password as "Not Vulnerable" when it still worked. Any scan result should be checked by hand before it is reported to leaders.
- **The Part 4 figures are sample data.** Earlier months are random numbers, and the current month is set to the ideal value by the script. The dashboard therefore shows how reporting looks, and not the real state of the lab. It also disagrees with the tracker about the number of risks (Section 7.2).
- **The tracker status colours are flawed** for "lower is better" metrics, as shown in Section 4.9.

10. Recommendations

If this were a real organisation, I would take the following steps, in this order:

- **Repair the real flaw.** Replace Struts 2.3.30 with version 2.3.32 or later (or 2.5.10.1 or later), then repeat the scan and the test against the new container to prove the fix.
- **Finish the database fix.** Check for other administrator accounts, replace the application user password, update the compose file so a rebuild does not bring back the weak password, store all secrets in a proper tool, and run the scanner again.
- **Make the scanner honest.** It should report "could not connect" separately from "login refused", so that a stopped container is never shown as a safe one.
- **Make the patch tool keep a full record.** It should read the existing status file and change only one entry, so that all three statuses are kept.
- **Separate the networks.** Put the web server and the database in different zones and allow only the connections that are needed.
- **Fix the dashboard logic.** Store whether higher or lower is better for each metric, feed the dashboard from real scan and audit data, and make it agree with the tracker.
- **Add the missing metrics.** Measure weak or default passwords and direct web-to-database paths, so that those controls can be watched (Section 6.6).
- **Make monitoring real.** Run the monitoring script on a schedule, send alerts to a named person and test that alarms are received.

- **Keep the lab running.** Set containers to restart automatically and check that they are running before every test.
- **Keep people accountable.** Give every policy, control and metric a named owner and report results to the board on a regular schedule.

11. Conclusion

This lab showed, in a safe and controlled way, how a small group of ordinary weaknesses can add up to a serious breach: a known flaw left unrepaired, a default password, a flat network and no live monitoring. Just as important, it showed that these weaknesses were allowed to stay because the rules, owners and checks of good governance were missing or not working.

I built the environment, scanned it, simulated an attack, analysed six governance failures, compared them with Equifax, added five groups of controls and produced a full set of reports for leaders. I also found and fixed several faults in the lab tools. The most valuable moment came from the database password: the scanner said it was safe, a direct test showed it was not, and only then could the fix be proved. That is exactly the habit that good governance needs, which is to check the evidence before reporting good news.

The main lesson I take away is that security governance is not only about tools. It is about making sure that policies are carried out, that numbers tell the truth and that someone is always responsible for acting on them.

Appendix A: Files produced during the lab

File	Size (kilobytes)	Purpose
docker-compose.yml	-	Describes the containers and the network
database_init/init.sql	-	Creates the customers table and sample records
vulnerability_scanner.py	7.0	Scans for the three weaknesses
vulnerability_report_20261007_145614.json	1.2	Final Part 1 scan report
patch_management.py	3.5	Lists and applies (simulated) patches
governance_tracker.py and initialize_governance.py	-	Records policies, controls, risks, incidents, metrics and audits
governance_data.json	4.6	Stored tracker data
simulate_attack.py and attack_log.json	0.2 (log)	Simulated attack and its log
analyze_governance_failures.py	-	Finds governance failures
governance_failure_report_20261008_054906.json	2.1	Six failures found
equifax_comparison.md	2.8	Comparison with the Equifax case
implement_governance_controls.py	-	Adds the five groups of controls
governance_controls_report.md	3.9	Documents the controls
governance_metrics.py	12	Creates metrics, charts, dashboard and board report
executive_dashboard.html	4.0	Executive dashboard web page
board_report.md	1.9	Board-level report
metrics_framework.md	3.0	Explains each metric
dashboard_user_guide.md	0.4	How to use the dashboard
metrics_visualizations (five charts)	about 223	Trend charts: patch_compliance.png, vuln_remediation.png, security_incidents.png, monitoring_coverage.png and policy_compliance.png

Appendix B: Timeline of the lab work

The times below are taken from file names and file listings in the screenshots.

Date and time	Event	Figure
7 October 2026, 14:27	vulnerability_scanner.py saved	43
7 October 2026, 14:30	First scan (report 143035) with two checks in error	44
7 October 2026, 14:56	Final Part 1 scan (report 145614)	54
8 October 2026, 04:03	patch_management.py saved	57
8 October 2026, 04:23	governance_data.json saved	69
8 October 2026, 05:49	Governance failure report created (name ends 054906)	90
8 October 2026, 05:54	equifax_comparison.md saved	93
8 October 2026, 06:18	First scan after the controls (report 061829)	97
8 October 2026, 06:25	governance_controls_report.md saved	105
8 October 2026, 06:36	Second verification scan (report 063609)	110
8 October 2026, 06:43 to 06:58	Metrics script saved, dashboard and board report created (06:45), metrics framework (06:55) and user guide (06:58)	114 and 128
Not timed	Database password tests (the screenshots show no clock)	100 to 103

Appendix C: Deliverables checklist

Part	Deliverable	Evidence in this report
Part 1	Working environment	Figures 26, 34 and 37
Part 1	Vulnerability scan report	Figures 53 and 54
Part 1	Governance tracker dashboard	Figures 67 and 68
Part 1	Explanation of the link to Equifax	Section 4.10
Part 2	Attack simulation and log	Figure 80
Part 2	Governance failure report	Figure 90 and Section 5.2
Part 2	Comparison with Equifax	Figures 91, 92 and 93
Part 3	Controls implemented	Figures 96 and 97
Part 3	Controls report	Figures 106, 107, 108 and 109
Part 3	Verification and password tests	Figures 98, 110, 99 and 100 to 103
Part 4	Executive dashboard	Figure 116
Part 4	Board report, metrics framework and user guide	Figures 123, 124, 125 and 126
Part 4	Five trend charts	Figures 118, 119, 120, 121 and 122

Appendix D: List of figures

Figure	Description
1	Creating the lab folder. The attempt to create the Docker network fails because Docker is not installed.
2	Installing the Docker command tool (docker-cli) after the "command not found" message.
3	The Docker command tool being downloaded and unpacked (22.4 megabytes).
4	Installing the full Docker engine package (docker.io) together with the packages it needs.
5	The Docker engine installation finishing.
6	Starting the Docker service and confirming that it is active and running.
7	The "hello-world" test container runs. Docker reports that the installation is working.
8	Docker commands now run correctly, and the lab network (secgov_network) is created.
9	The end of docker-compose.yml: the monitoring container and the network setting, closed with the EOF marker.
10	Writing init.sql: the customers table and its columns, including social security number and card number.
11	The three made-up customer records added to the table.
12	Checking that init.sql exists inside the database_init folder.
13	The lab folder now contains database_init, monitoring and docker-compose.yml.
14	Creating the database_init and monitoring folders and opening init.sql in the nano editor.
15	Reviewing init.sql in the nano editor.
16	Reviewing the monitoring section of docker-compose.yml in the nano editor.
17	The compose check fails: the old command is not found, the new command is unknown and the plug-in package is not available.
18	Downloading Docker Compose version 2 as a standalone file into Docker's plug-in folder.
19	The tool is made executable and now reports its version. The configuration check prints the database service and a warning about the out-of-date "version" line.
20	The rest of the configuration check: the monitoring and web services. The web service still points to the image named in the lab sheet (vulhub/struts2:s2-045).
21	The docker group appears in the user's groups, and docker ps runs without administrator rights.
22	The vulhub project folder for s2-045. Its own docker-compose.yml uses the image vulhub/struts2:2.3.30 and publishes port 8080.
23	Pulling the image vulhub/struts2:2.3.30.
24	The docker-compose command is not found, so the docker-compose package is installed from the Kali software list.
25	The docker-compose package finishes installing.

Figure	Description
26	The Struts container starts. After a mistyped option is corrected, docker ps shows the container running.
27	The Struts test application opened in the browser on the virtual machine: the "Fileupload sample" page.
28	Folder listing and docker ps -a: the Struts container is up and running, next to the older hello-world test container.
29	Moving between folders to return to the main lab folder.
30	Creating secgov_network and listing the Docker networks. The new network sits beside the default one created for the Struts container.
31	A formatted container list shows the Struts container, and docker inspect starts to show its networks.
32	Docker inspect output: the Struts container has an address on its own network (172.18.0.2) and another on secgov_network (172.19.0.2).
33	Starting the database and monitoring containers. Docker pulls the images and starts both.
34	All three containers are running: database_server, monitoring_server and the Struts web server.
35	The first part of the database log: MySQL starting up.
36	More of the database log, including warnings about an outdated encryption setting.
37	Logging in as app_user shows the customers table. The network inspection begins: subnet 172.19.0.0/16.
38	The network inspection continues: monitoring_server is at 172.19.0.3 and database_server is at 172.19.0.4.
39	A listing of the files in the lab folder, including the vulhub project and database_init/init.sql.
40	Opening vulnerability_scanner.py in the nano editor.
41	The top of the scanner program: the import lines.
42	The end of the scanner program: the summary printout and the main section.
43	The scanner file is saved (6,998 bytes).
44	Syntax check, making the file executable and the first scan. The Struts check shows an error.
45	First scan results: Struts check shows an error (name not found), the database is Vulnerable and the network check shows an error (decoding problem).
46	Inspecting secgov_network to find the container addresses.
47	The rest of the inspection output, showing each container and its address.
48	The scanner settings now use the address http://172.19.0.2:8080 for the Struts check.
49	Second scan: Struts is now "Potentially Vulnerable" and the database is Vulnerable.
50	The rest of the second scan: the network check still shows a decoding error.
51	Searching the scanner for the lines where the checks are called.

Figure	Description
52	The network check in the editor: the text option is removed so binary data can be read safely.
53	The final scan: Struts Critical (Potentially Vulnerable), database High (Vulnerable) and network Medium (Vulnerable).
54	The saved scan report file (1.2 kilobytes).
55	The patch tool in the nano editor: the list of three available patches.
56	The end of the patch tool: the options --list and --apply.
57	The file is saved (3.5 kilobytes) and its syntax is checked.
58	Listing the three available patches with their severity and affected system.
59	Applying PATCH-001, PATCH-002 and PATCH-003 one after the other. Each reports success.
60	The status file contains only {"network_segmented": true}.
61	The governance tracker program in the nano editor: the GovernanceTracker class.
62	The end of the tracker: the dashboard option and the main section.
63	Creating and checking the tracker, then starting the initialisation script.
64	The initialisation script adds the first policies.
65	The end of the initialisation script: a metric, an audit and the final message.
66	Running the initialisation script. It confirms: "Governance tracker initialized with sample data".
67	The tracker dashboard output: 3 policies, 3 controls, 3 risks, 1 incident, 3 metrics and 1 audit, with 1 critical, 1 high and 1 medium risk.
68	The metric status lines. Patch Compliance is Red (85 against a target of 95).
69	The saved tracker data file (4.6 kilobytes).
70	Opening simulate_attack.py in the nano editor.
71	The top of the attack script. The first version aims at http://172.19.0.2:8080.
72	The end of the attack script: it writes the attack log and handles errors.
73	Checking the script, fixing a permission problem and a typing mistake, and the first run, which fails to connect.
74	docker ps: all three containers are running.
75	curl http://localhost:8080 returns the Struts "Fileupload sample" page. The server is working.
76	The rest of the page returned by curl.
77	The attack script fails twice with "Connection refused" for 172.19.0.2:8080.
78	Opening simulate_attack.py again to change the target address.
79	The target is changed to http://localhost:8080.

Figure	Description
80	The attack simulation completes and saves its log. Struts S2-045 is detected, and data theft is simulated.
81	Opening analyze_governance_failures.py.
82	The top of the analysis script: it loads the latest vulnerability report.
83	The end of the analysis script: it writes the failure report and prints a summary.
84	First run of the analysis script: a permission problem is fixed, and the run then stops with an error.
85	My vulnerability report printed in full. The findings are stored under "results".
86	The rest of the report: the network segmentation finding and its advice.
87	Fix 1: the script now reads "results" instead of "vulnerabilities".
88	Second run: another error, where the script reads a summary count that my report does not have.
89	Fix 2: the script now builds its own list of Critical findings from "results" and counts them.
90	The analysis completes: 6 governance failures identified, one in each of six categories.
91	The comparison report: overview and the key governance failures at Equifax.
92	The comparison table and the lessons learned.
93	The comparison report is saved (2.8 kilobytes).
94	Creating the controls script, checking its syntax and making it executable.
95	The controls script in the nano editor: the patch management control.
96	Running the script: patch management (CTL-004) and network segmentation controls are applied.
97	The oversight policy (POL-004) and control (CTL-008) are added, and the verification begins.
98	The scanner re-run after the controls: Struts shows an error, the database shows Not Vulnerable and network segmentation shows Potentially Vulnerable.
99	The second attack simulation ends with "Connection refused" for localhost on port 8080.
100	docker ps lists no running containers, so database_server is started by hand. A direct login with the default password then succeeds and returns 1.
101	The default administrator password is changed with an ALTER USER command. Trying the default password straight afterwards is rejected with "Access denied".
102	Logging in with the password typed at the prompt. The first two attempts are rejected with "Access denied". The third is accepted and returns 1.
103	The default password is tested again and is rejected with "Access denied".
104	Writing governance_controls_report.md in the nano editor.
105	The controls report is saved (3.9 kilobytes).
106	The controls report displayed: overview and patch management controls.

Figure	Description
107	The controls report continues: security monitoring and governance oversight controls.
108	The end of the controls report: verification results, tracker totals and the conclusion.
109	The deliverables from Parts 2 and 3: attack_log.json, equifax_comparison.md, governance_controls_report.md and the governance failure report.
110	A second verification run shows the same results, and a new scan report (vulnerability_report_20261008_063609.json).
111	The first part of governance_metrics.py: the function that creates six months of sample data.
112	The end of the metrics script: it writes board_report.md and runs all the steps.
113	Checking the script, making it executable and running it. It creates the dashboard, the board report and five charts.
114	The files created: board_report.md, executive_dashboard.html and the metrics_visualizations folder with five charts.
115	Opening the dashboard in the web browser with the xdg-open command.
116	The Security Governance Executive Dashboard in the browser: posture, risk and compliance panels and key recommendations.
117	The five chart files in the metrics_visualizations folder, shown in the file manager.
118	Patch Compliance Trend: about 71 to 81 per cent for five months, then 100 per cent in October 2026, above the 95 per cent target.
119	Vulnerability Remediation Time Trend: about 11 to 15 days for five months, above the 7-day target, then 1 day in October 2026.
120	Security Incidents Trend: 3 incidents in May and June, 1 from July to September, then 0 in October 2026, which meets the target.
121	Security Monitoring Coverage Trend: about 70 to 79 per cent for five months, then 100 per cent in October 2026.
122	Policy Compliance Trend: about 76 to 86 per cent for five months, then 100 per cent in October 2026.
123	The board report: executive summary, key metrics (all Green) and the risk summary.
124	The rest of the board report: security improvements, recommendations and the conclusion.
125	The metrics framework: purpose and the table of key metrics with targets, frequency, owner and audience.
126	The dashboard user guide: its purpose and how to open the dashboard.
127	Creating the board report, metrics framework and user guide in the nano editor, and checking their sizes.
128	All the Part 4 files: the metrics script, dashboard, board report, metrics framework and user guide.